
Learned Robust PCA: A Scalable Deep Unfolding Approach for High-Dimensional Outlier Detection

HanQin Cai*

Department of Mathematics
University of California, Los Angeles
Los Angeles, CA 90095, USA
hqcai@math.ucla.edu

Jialin Liu

Decision Intelligence Lab
Damo Academy, Alibaba US
Bellevue, WA 98004, USA
jialin.liu@alibaba-inc.com

Wotao Yin

Decision Intelligence Lab
Damo Academy, Alibaba US
Bellevue, WA 98004, USA
wotao.yin@alibaba-inc.com

Abstract

Robust principal component analysis (RPCA) is a critical tool in modern machine learning, which detects outliers in the task of low-rank matrix reconstruction. In this paper, we propose a *scalable* and *learnable* non-convex approach for high-dimensional RPCA problems, which we call Learned Robust PCA (LRPCA). LRPCA is highly efficient, and its free parameters can be effectively learned to optimize via deep unfolding. Moreover, we extend deep unfolding from finite iterations to infinite iterations via a novel feedforward-recurrent-mixed neural network model. We establish the recovery guarantee of LRPCA under mild assumptions for RPCA. Numerical experiments show that LRPCA outperforms the state-of-the-art RPCA algorithms, such as ScaledGD and AltProj, on both synthetic datasets and real-world applications.

1 Introduction

Over the last decade, robust principal component analysis (RPCA), one of the fundamental dimension reduction techniques, has received intensive investigations from theoretical and empirical perspectives [1–13]. RPCA also plays a key role in a wide range of machine learning tasks, such as video background subtraction [14], singing-voice separation [15], face modeling [16], image alignment [17], feature identification [18], community detection [19], fault detection [20], and NMR spectrum recovery [21]. While the standard principal component analysis (PCA) is known for its high sensitivity to outliers, RPCA is designed to enhance the robustness of PCA when outliers are present. In this paper, we consider the following RPCA setting: given an observed corrupted data matrix

$$Y = X_* + S_* \in \mathbb{R}^{n_1 \times n_2}, \quad (1)$$

where X_* is a rank- r data matrix and S_* is a sparse outlier matrix, reconstruct X_* and S_* simultaneously from Y .

One of the main challenges of designing an RPCA algorithm is to avoid high computational costs. Inspired by deep unfolded sparse coding [22], some recent works [23–25] have successfully extended deep unfolding techniques to RPCA and achieved noticeable accelerations in certain applications.

*Authors are listed in alphabetic order. Correspondence shall be addressed to H.Q. Cai.

Specifically, by parameterizing a classic RPCA algorithm and unfolding it as a feedforward neural network (FNN), one can improve its performance by learning the parameters of the FNN through backpropagation. Since RPCA problems of a specific application often share similar key properties (e.g., rank, incoherence, and amount of outliers), the parameters learned from training examples that share the properties can lead to superior performance. Nevertheless, all existing learning-based approaches call an expensive step of singular value thresholding (SVT) [26] at every iteration during both training and inference. SVT involves a full or truncated singular value decomposition (SVD), which costs from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^2r)$ flops² with some large hidden constant in front³. Thus, these approaches are not scalable to high-dimensional RPCA problems. Another issue is that the existing approaches only learn the parameters for a finite number of iterations. If a user targets at a specific accuracy of recovery, then the prior knowledge of the unfolded algorithm is required for choosing the number of unfolded layers. Moreover, if the user desires better accuracy later, one may have to restart the learning process to obtain parameters of the extra iterations.

We must ask the questions: “*Can one design a highly efficient and easy-to-learn method for high-dimensional RPCA problems?*” and “*Do we have to restrict ourselves to finite-iteration unfolding (or a fixed-layer neural network)?*” In this paper, we aim to answer these questions by proposing a *scalable* and *learnable* approach for high-dimensional RPCA problems, which has a flexible feedforward-recurrent-mixed neural network model for potentially infinite-iteration unfolding. Consequently, our approach can satisfy arbitrary accuracy without relearning the parameters.

Related work. The earlier works [1–3] for RPCA are based on the convex model:

$$\underset{\mathbf{X}, \mathbf{S}}{\text{minimize}} \|\mathbf{X}\|_* + \lambda \|\mathbf{S}\|_1, \quad \text{subject to } \mathbf{Y} = \mathbf{X} + \mathbf{S}. \quad (2)$$

It has been shown that (2) guarantees exact recovery, provided $\alpha \lesssim \mathcal{O}(1/\mu r)$, a condition with an optimal order [4, 5]. Therein, α is the sparsity parameter of $\mathbf{S}_*$ and μ is the incoherence parameter of $\mathbf{L}_*$, which will be formally defined later in Assumptions 2 and 1, respectively. Problem (2) can be transformed into a semidefinite program and has a variety of dedicated methods, their per-iteration complexities are at least $\mathcal{O}(n^3)$ flops and some of them guarantee only sublinear convergence.

Later, various non-convex methods with linear convergence were proposed for RPCA: [7] uses alternating minimization, and it can tolerate outliers up to a fraction of only $\alpha \lesssim \mathcal{O}(1/\mu^{2/3}r^{2/3}n)$. [6] develops an alternating projections method (AltProj) that alternatively projects $\mathbf{Y} - \mathbf{S}$ onto the space of low-rank matrices and $\mathbf{Y} - \mathbf{X}$ onto the space of sparse matrices. AltProj costs $\mathcal{O}(n^2r^2)$ flops and tolerates outliers up to $\alpha \lesssim \mathcal{O}(1/\mu r)$. An accelerated version of AltProj (AccAltProj) was proposed in [10], which improves the computational complexity to $\mathcal{O}(n^2r)$ with sacrifices on the tolerance to outliers, which are allowed up to $\alpha \lesssim \mathcal{O}(1/\max\{\mu r^2\kappa^3, \mu^{3/2}r^2\kappa, \mu^2r^2\})$, where κ is the condition number of $\mathbf{X}_*$. Note that the key technique used for outlier detection in AltProj and AccAltProj is employing adaptive hard-thresholding parameters.

Another line of non-convex RPCA algorithms reformulate the low-rank component as $\mathbf{X} = \mathbf{L}\mathbf{R}^\top$, where $\mathbf{L} \in \mathbb{R}^{n_1 \times r}$ and $\mathbf{R} \in \mathbb{R}^{n_2 \times r}$, and then performs gradient descend on $\mathbf{L}$ and $\mathbf{R}$ separately. Since the low-rank constraint of $\mathbf{X}$ is automatically satisfied under this reformulation, the costly step of SVD is avoided (except for one SVD during initialization). Specifically speaking, [8] uses a complicated objective function, which includes a practically unimportant balance regularization $\|\mathbf{L}^\top \mathbf{L} - \mathbf{R}^\top \mathbf{R}\|_F$. While GD costs only $\mathcal{O}(n^2r)$ flops per iteration, its convergence rate relies highly on κ . Recently, ScaledGD [12] introduces a scaling term in gradient descend steps to remove the dependence on κ in the convergence rate of GD. ScaledGD also drops the balance regularization and targets on a more elegant objective:

$$\underset{\mathbf{L} \in \mathbb{R}^{n_1 \times r}, \mathbf{R} \in \mathbb{R}^{n_2 \times r}, \mathbf{S} \in \mathbb{R}^{n_1 \times n_2}}{\text{minimize}} \quad \frac{1}{2} \|\mathbf{L}\mathbf{R}^\top + \mathbf{S} - \mathbf{Y}\|_F^2, \quad \text{subject to } \mathbf{S} \text{ is } \alpha\text{-sparse}, \quad (3)$$

where α -sparsity means no more than a fraction α of non-zeros on every row and every column of $\mathbf{S}$. To enforce the sparsity constraint of $\mathbf{S}$, ScaledGD (and also GD) employs a sparsification operator:

$$[\mathcal{T}_\alpha(\mathbf{M})]_{i,j} = \begin{cases} [\mathbf{M}]_{i,j}, & \text{if } |[\mathbf{M}]_{i,j}| \geq |[\mathbf{M}]_{i,:}^{(\tilde{\alpha}n_2)}| \text{ and } |[\mathbf{M}]_{i,j}| \geq |[\mathbf{M}]_{:,j}^{(\tilde{\alpha}n_1)}| \\ 0, & \text{otherwise} \end{cases}, \quad (4)$$

²For ease of presentation, we take $n := n_1 = n_2$ when discuss computational complexities.

³The hidden constant is often hundreds and sometimes thousands.

where $[\cdot]_{i,:}^{(k)}$ and $[\cdot]_{:,j}^{(k)}$ denote the k -th largest element in magnitude on the i -th row and in the j -th column, respectively. In other words, $\mathcal{T}_{\tilde{\alpha}}$ keeps only the largest $\tilde{\alpha}$ fraction of the entries on every row and in every column. The per-iteration computational complexity of ScaledGD remains $\mathcal{O}(n^2r)$ and it converges faster on ill-conditioned problems. The tolerance to outliers of GD and ScaledGD are $\alpha \lesssim \mathcal{O}(1/\max\{\mu r^{3/2}\kappa^{3/2}, \mu r\kappa^2\})$ and $\alpha \lesssim \mathcal{O}(1/\mu r^{3/2}\kappa)$, respectively.

It is worth mentioning there exist some other RPCA settings that high-dimensional problems can be efficiently solved. For example, if the columns of X_* are independently sampled over a zero-mean multivariate Gaussian distribution, an efficient approach is using Grassmann averages [27, 28].

Deep unfolding is a technique that dates back to 2010 from a fast approximate method for LASSO: LISTA [22] parameterizes the classic Iterative Shrinkage-Thresholding Algorithm (ISTA) as a fully-connected feedforward neural network and demonstrates that the trained neural network generalizes well to unseen samples from the distribution used for training. It achieves the same accuracy within one or two order-of-magnitude fewer iterations than the original ISTA. Later works [29–36] extend this approach to different problems and network architectures and have good performance. Another recently developed technique “learning to learn” [37–39] parameterizes iterative algorithms as recurrent neural networks and shows great potential on machine learning tasks.

Applying deep unfolding on RPCA appeared recently. CORONA [23, 24] uses convolutional layers in deep unfolding and focuses on the application of ultrasound imaging. It uses SVT for low-rank approximation and mixed $\ell_{1,2}$ thresholding for outlier detection since the outliers in ultrasound imaging problems are usually structured. refRPCA [25] focuses on video foreground-background separation and also uses SVT for low-rank approximation. However, refRPCA employs a sequence of constructed reference frames to reflex correlation between consecutive frames in the video, which leads to a complicated yet learnable proximal operator for outlier detection. Given the high computational cost of SVT, both CORONA and refRPCA are not scalable. In addition, Denise [40] studies a deep unfolding RPCA method for the special case of positive semidefinite low-rank matrices. Denise achieves a remarkable speedup from baselines; however, such RPCA applications are limited.

Contribution. In this work, we propose a novel learning-based method, which we call Learned Robust PCA (LRPCA), for solving high-dimensional RPCA problems. Our main contributions are:

1. The proposed algorithm, LRPCA, is *scalable* and *learnable*. It uses a simple formula and differentiable operators to avoid the costly SVD and partial sorting (see Algorithm 1). LRPCA costs $\mathcal{O}(n^2r)$ flops with a small constant⁴ while all its parameters can be optimized by training.
2. An exact recovery guarantee is established for LRPCA under some mild conditions (see Theorem 1). In particular, LRPCA can tolerate outliers up to a fraction of $\alpha \lesssim \mathcal{O}(1/\mu r^{3/2}\kappa)$. Moreover, the theorem confirms that there exist a set of parameters for LRPCA to outperform the baseline algorithm ScaledGD.
3. We proposed a novel feedforward-recurrent-mixed neural network (FRMNN) model to solve RPCA. The new model first unfolds finite iterations of LRPCA and individually learns the parameters at the significant iteration; then, it learns the rules of parameter updating for subsequent iterations. Therefore, FRMNN learns the parameters for infinite iterations of LRPCA while we ensure no performance reduction from classic deep unfolding.
4. The numerical experiments confirm the advantages of LRPCA on both synthetic and real-world datasets. In particular, we successfully apply LRPCA to large video background subtraction tasks where the problem dimensions exceed the capacities of the existing learning-based RPCA approaches.

Notation. For any matrix M , $[M]_{i,j}$ denotes the (i, j) -th entry, $\sigma_i(M)$ denotes the i -th singular value, $\|M\|_1 := \sum_{i,j} |[M]_{i,j}|$ denotes the entrywise ℓ_1 norm, $\|M\|_F := (\sum_{i,j} [M]_{i,j}^2)^{1/2}$ denotes the Frobenius norm, $\|M\|_* := \sum_i \sigma_i(M)$ denotes the nuclear norm, $\|M\|_\infty := \max_{i,j} |[M]_{i,j}|$ denotes the largest magnitude, $\|M\|_{2,\infty} := \max_i (\sum_j [M]_{i,j}^2)^{1/2}$ denotes the largest row-wise ℓ_2 norm, and $M^\top$ denotes the transpose. We use $\kappa := \frac{\sigma_1(X_*)}{\sigma_r(X_*)}$ to denote the condition number of X_* .

⁴More precisely, LRPCA costs as low as $3n^2r + 3n^2 + \mathcal{O}(nr^2)$ flops per iteration, which is much smaller than the hidden constant of truncated SVD.

2 Proposed method

In this section, we first describe the proposed learnable algorithm and then present the recovery guarantee.

We consider the non-convex minimization problem:

$$\underset{\mathbf{L} \in \mathbb{R}^{n_1 \times r}, \mathbf{R} \in \mathbb{R}^{n_2 \times r}, \mathbf{S} \in \mathbb{R}^{n_1 \times n_2}}{\text{minimize}} \quad \frac{1}{2} \|\mathbf{L}\mathbf{R}^\top + \mathbf{S} - \mathbf{Y}\|_F^2, \quad \text{subject to } \text{supp}(\mathbf{S}) \subseteq \text{supp}(\mathbf{S}_*). \quad (5)$$

One may find (5) similar to the objective (3) of ScaledGD but with a different sparsity constraint for $\mathbf{S}$. The user may not know the true support of $\mathbf{S}_*$. Also, the two constraints in (3) and (5) may seem somewhat equivalent when $\mathbf{S}_*$ is assumed α -sparse. However, we emphasize that our algorithm design does not rely on an accurate estimation of α , and our method eliminates false-positives at every outlier-detection step.

Our algorithm proceeds in two phases: initialization and iterative updates. The first phase is done by a modified spectral initialization. In the second phase, we iteratively update outlier estimates via soft-thresholding and the factorized low-rank component estimates via scaled gradient descends. All the parameters of our algorithm are learnable during deep unfolding. The proposed algorithm, LRPCA, is summarized in Algorithm 1. We now discuss the key details of LRPCA and begin with the second phase.

Algorithm 1 Learned Robust PCA (LRPCA)

```

1: Input:  $\mathbf{Y} = \mathbf{X}_* + \mathbf{S}_*$ : sparsely corrupted matrix;  $r$ : the rank of underlying low-rank matrix;
    $\{\zeta_k\}$ : a set of learned thresholding values;  $\{\eta_k\}$ : a set of learned step sizes.
2: // Initialization:
3:  $\mathbf{S}_0 = \mathcal{S}_{\zeta_0}(\mathbf{Y})$ 
4:  $[\mathbf{U}_0, \mathbf{\Sigma}_0, \mathbf{V}_0] = \text{SVD}_r(\mathbf{Y} - \mathbf{S}_0)$ 
5:  $\mathbf{L}_0 = \mathbf{U}_0 \mathbf{\Sigma}_0^{1/2}$ ,  $\mathbf{R}_0 = \mathbf{V}_0 \mathbf{\Sigma}_0^{1/2}$ 
6: // Iterative updates:
7: while Not(Stopping Condition) do
8:    $\mathbf{S}_{k+1} = \mathcal{S}_{\zeta_{k+1}}(\mathbf{Y} - \mathbf{L}_k \mathbf{R}_k^\top)$ 
9:    $\mathbf{L}_{k+1} = \mathbf{L}_k - \eta_{k+1}(\mathbf{L}_k \mathbf{R}_k^\top + \mathbf{S}_{k+1} - \mathbf{Y}) \mathbf{R}_k (\mathbf{R}_k^\top \mathbf{R}_k)^{-1}$ 
10:   $\mathbf{R}_{k+1} = \mathbf{R}_k - \eta_{k+1}(\mathbf{L}_k \mathbf{R}_k^\top + \mathbf{S}_{k+1} - \mathbf{Y})^\top \mathbf{L}_k (\mathbf{L}_k^\top \mathbf{L}_k)^{-1}$ 
11: end while
12: Output:  $\mathbf{X}_K = \mathbf{L}_K \mathbf{R}_K^\top$ : the recovered low-rank matrix.

```

Updating $\mathbf{S}$. In ScaledGD [12], the sparse outlier matrix is updated by $\mathbf{S}_{k+1} = \mathcal{T}_{\tilde{\alpha}}(\mathbf{Y} - \mathbf{L}_k \mathbf{R}_k^\top)$, where the sparsification operator $\mathcal{T}_{\tilde{\alpha}}$ is defined in (4). Note that $\mathcal{T}_{\tilde{\alpha}}$ requires an accurate estimate of α and its execution involves partial sorting on each row and column. Moreover, deep unfolding and parameter learning cannot be applied to $\mathcal{T}_{\tilde{\alpha}}$ since it is not differentiable. Hence, we hope to find an efficient and effectively learnable operator to replace $\mathcal{T}_{\tilde{\alpha}}$.

The well-known soft-thresholding, a.k.a. shrinkage operator,

$$[\mathcal{S}_{\zeta}(\mathbf{M})]_{i,j} = \text{sign}([\mathbf{M}]_{i,j}) \cdot \max\{0, |[\mathbf{M}]_{i,j}| - \zeta\} \quad (6)$$

is our choice. Soft-thresholding has been applied as a proximal operator of ℓ_1 norm (to a matrix entrywise) in some RPCA algorithms [2]. However, a fixed threshold, which is determined by the regularization parameter, leads to only sublinear convergence of the algorithm. Inspired by AltProj, LISTA, and their followup works [6, 10, 22, 29–34, 36], we seek for a set of thresholding parameters $\{\zeta_k\}$ that let our algorithm outperform the baseline ScaledGD.

In fact, we find that the simple soft-thresholding:

$$\mathbf{S}_{k+1} = \mathcal{S}_{\zeta_{k+1}}(\mathbf{Y} - \mathbf{L}_k \mathbf{R}_k^\top) \quad (7)$$

can also give a linear convergence guarantee if we carefully choose the thresholds, and the selected $\{\zeta_k\}$ also guarantees $\text{supp}(\mathbf{S}_k) \subseteq \text{supp}(\mathbf{S}_*)$ at every iteration, i.e., no false-positive outliers. Moreover, the proposed algorithm with selected thresholds can converge even faster than ScaledGD under the very same assumptions (see Theorem 1 below for a formal statement.) Nevertheless, the theoretical selection of $\{\zeta_k\}$ relies on the knowledge of $\mathbf{X}_*$, which is usually unknown to the user. Fortunately, these thresholds can be reliably learned using the deep unfolding techniques.

Updating $\mathbf{X}$. To avoid the low-rank constraint on $\mathbf{X}$, we write a rank- r matrix as product of a tall and a fat matrices, that is, $\mathbf{X} = \mathbf{L}\mathbf{R}^\top \in \mathbb{R}^{n_1 \times n_2}$ with $\mathbf{L} \in \mathbb{R}^{n_1 \times r}$ and $\mathbf{R} \in \mathbb{R}^{n_2 \times r}$. Denote the loss function $f_k := f(\mathbf{L}_k, \mathbf{R}_k) := \frac{1}{2} \|\mathbf{L}_k \mathbf{R}_k^\top + \mathbf{S} - \mathbf{Y}\|_{\text{F}}^2$. The gradients can be easily computed:

$$\nabla_{\mathbf{L}} f_k = (\mathbf{L}_k \mathbf{R}_k^\top + \mathbf{S} - \mathbf{Y}) \mathbf{R}_k \quad \text{and} \quad \nabla_{\mathbf{R}} f_k = (\mathbf{L}_k \mathbf{R}_k^\top + \mathbf{S} - \mathbf{Y})^\top \mathbf{L}_k. \quad (8)$$

We could apply a step of gradient descent on $\mathbf{L}$ and $\mathbf{R}$. However, [12] finds the vanilla gradient descent approach suffers from ill-conditioning and thus introduces the scaled terms $(\mathbf{R}_k^\top \mathbf{R}_k)^{-1}$ and $(\mathbf{L}_k^\top \mathbf{L}_k)^{-1}$ to overcome this weakness. In particular, we follow ScaledGD and update the low-rank component:

$$\begin{aligned} \mathbf{L}_{k+1} &= \mathbf{L}_k - \eta_{k+1} \nabla_{\mathbf{L}} f_k \cdot (\mathbf{R}_k^\top \mathbf{R}_k)^{-1}, \\ \mathbf{R}_{k+1} &= \mathbf{R}_k - \eta_{k+1} \nabla_{\mathbf{R}} f_k \cdot (\mathbf{L}_k^\top \mathbf{L}_k)^{-1}, \end{aligned} \quad (9)$$

where η_{k+1} is the step size at the $(k+1)$ -th iteration.

Initialization. We use a modified spectral initialization with soft-thresholding in the proposed algorithm. That is, we first initialize the sparse matrix by $\mathbf{S}_0 = \mathcal{S}_{\zeta_0}(\mathbf{Y})$, which should remove the obvious outliers. Next, for the low-rank component, we take $\mathbf{L}_0 = \mathbf{U}_0 \Sigma_0^{1/2}$ and $\mathbf{R}_0 = \mathbf{V}_0 \Sigma_0^{1/2}$, where $\mathbf{U}_0 \Sigma_0 \mathbf{V}_0^\top$ is the best rank- r approximation (denoted by SVD_r) of $\mathbf{Y} - \mathbf{S}_0$. Clearly, the initial thresholding step is crucial for the quality of initialization and the thresholding parameter ζ_0 can be optimized through learning.

For the huge-scale problems where even a single truncated SVD is computationally prohibitive, one may replace the SVD step in initialization by some batch-based low-rank approximations, e.g., CUR decomposition. While its stability lacks theoretical support when outliers appear, the empirical evidence shows that a single CUR decomposition can provide sufficiently good initialization [11].

Computational complexity. Along with guaranteed linear convergence, LRPCA costs only $3n^2r + 3n^2 + \mathcal{O}(nr^2)$ flops per iteration provided $r \ll n$. In contrast, truncated SVD costs $\mathcal{O}(n^2r)$ flops with a much larger hidden constant. The breakdown of LRPCA's complexity is provided in Appendix B.

Limitations. LRPCA assumes the knowledge of $\text{rank}(\mathbf{X}_*)$, which is commonly assumed in many non-convex matrix recovery algorithms. In fact, the rank of $\mathbf{X}_*$ is fixed or can be easily obtained from prior knowledge in many applications, e.g., Euclidean distance matrices are rank- $(d+2)$ for the system of d -dimensional points [41]. However, the true rank may be hard to get in other applications, and LRPCA is not designed to learn the true rank from training data. Thus, LRPCA may have reduced performance in such applications when the target rank is seriously misestimated.

2.1 Theoretical results

We present the recovery guarantee of LRPCA in this subsection. Moreover, when the parameters are selected correctly, we show that LRPCA provably outperforms the state-of-the-arts.

We start with two common assumptions for RPCA:

Assumption 1 (μ -incoherence of low-rank component). $\mathbf{X}_* \in \mathbb{R}^{n_1 \times n_2}$ is a rank- r matrix with μ -incoherence, i.e.,

$$\|\mathbf{U}_*\|_{2,\infty} \leq \sqrt{\frac{\mu r}{n_1}} \quad \text{and} \quad \|\mathbf{V}_*\|_{2,\infty} \leq \sqrt{\frac{\mu r}{n_2}} \quad (10)$$

for some constant $1 \leq \mu \leq n$, where $\mathbf{U}_* \Sigma_* \mathbf{V}_*^\top$ is the compact SVD of $\mathbf{X}_*$.

Assumption 2 (α -sparsity of outlier component). $\mathbf{S}_* \in \mathbb{R}^{n_1 \times n_2}$ is an α -sparse matrix, i.e., there are at most α fraction of non-zero elements in each row and column of $\mathbf{S}_*$. In particular, we require $\alpha \lesssim \mathcal{O}(\frac{1}{\mu r^{3/2} \kappa})$ for the guaranteed recovery, which matches the requirement for ScaledGD. We future assume the problem is well-posed, i.e., $\mu, r, \kappa \ll n$.

Remark 1. Note that some applications may have a more specific structure for outliers, which may lead to a more suitable operator for $\mathbf{S}$ updating in the particular applications. This work aims to solve the most common RPCA model with a sparsity assumption. Nevertheless, our learning framework can be adapted to another operator as long as it is differentiable.

By the rank-sparsity uncertainty principle, a matrix cannot be incoherent and sparse simultaneously [42]. The above two assumptions ensure the uniqueness of the solution in RPCA. Now, we are ready to present our main theorem:

Theorem 1 (Guaranteed recovery). *Suppose that $\mathbf{X}_\star$ is a rank- r matrix with μ -incoherence and $\mathbf{S}_\star$ is an α -sparse matrix with $\alpha \leq \frac{1}{10^4 \mu r^{3/2} \kappa}$. If we set the thresholding values $\zeta_0 = \|\mathbf{X}_\star\|_\infty$ and $\zeta_k = \|\mathbf{L}_{k-1} \mathbf{R}_{k-1}^\top - \mathbf{X}_\star\|_\infty$ for $k \geq 1$ for LRPCA, the iterates of LRPCA satisfy*

$$\|\mathbf{L}_k \mathbf{R}_k^\top - \mathbf{X}_\star\|_F \leq 0.03(1 - 0.6\eta)^k \sigma_r(\mathbf{X}_\star) \quad \text{and} \quad \text{supp}(\mathbf{S}_k) \subseteq \text{supp}(\mathbf{S}_\star), \quad (11)$$

with the step sizes $\eta_k = \eta \in [\frac{1}{4}, \frac{8}{9}]$.

Proof. The proof of Theorem 1 is deferred to Appendix A. $\square$

Essentially, Theorem 1 states that there exists a set of selected thresholding values $\{\zeta_k\}$ that allows one to replace the sparsification operator $\mathcal{T}_{\tilde{\alpha}}$ in ScaledGD with the simpler soft-thresholding operator $\mathcal{S}_\zeta$ and maintains the same linear convergence rate $1 - 0.6\eta$ under the same assumptions. Note that the theoretical choice of parameters relies on the knowledge of $\mathbf{X}_\star$ —an unknown factor. Thus, Theorem 1 can be read as a proof for the existence of the appropriate parameters.

Moreover, Theorem 1 shows two advantages of LRPCA:

1. Under the very same assumptions and constants, we allow the step sizes η_k to be as large as $\frac{8}{9}$; in contrast, ScaledGD has the step sizes no larger than $\frac{2}{3}$ [12, Theorem 2]. That is, LRPCA can provide faster convergence under the same sparsity condition, by allowing larger step sizes.
2. With the selected thresholding values, $\mathcal{S}_\zeta$ in LRPCA is effectively a projection onto $\text{supp}(\mathbf{S}_\star)$, which matches our sparsity constrain in objective function (5). That is, $\mathcal{S}_\zeta$ takes out the larger outliers, leaves the smaller outliers, and preserves all good entries — no false-positive outlier in $\mathbf{S}_k$. In contrast, $\mathcal{T}_{\tilde{\alpha}}$ necessarily yields some false-positive outliers in the earlier stages of ScaledGD, which drag the algorithm when outliers are relatively small.

Technical innovation. The main challenge to our analysis is to show both the distance error metric (i.e., $\text{dist}(\mathbf{L}_k, \mathbf{R}_k; \mathbf{L}_\star, \mathbf{R}_\star)$, later defined in Appendix A) and ℓ_∞ error metric (i.e., $\|\mathbf{L}_k \mathbf{R}_k^\top - \mathbf{X}_\star\|_\infty$) are linearly decreasing. While the former takes some minor modifications from the proof of ScaledGD, the latter is rather challenging and utilizes several new technical lemmata. Note that ScaledGD shows $\|\mathbf{L}_k \mathbf{R}_k^\top - \mathbf{X}_\star\|_\infty$ is always bounded but not necessary decreasing, which is insufficient for LRPCA.

3 Parameter learning

Theorem 1 shows the existence of “good” parameters $\{\zeta_k\}, \{\eta_k\}$ and, in this section, we describe how to obtain such parameters via machine learning techniques.

Feed-forward neural network. Classic deep unfolding methods unroll an iterative algorithm and truncates it into a fixed number, says K , iterations. Applying such idea to our model, we regard each iteration of Algorithm 1 as a layer of a neural network and regard the variables $\mathbf{L}_k, \mathbf{R}_k, \mathbf{S}_k$ as the units of the k -th hidden layer. The top part of Figure 1 demonstrates the structure of such feed-forward neural network (FNN). The k -th layer is denoted as $\mathcal{L}_k$. Based on (7) and (9), it takes $\mathbf{Y}$ as an input and has two parameters ζ_k, η_k when $k \geq 1$. The initial layer $\mathcal{L}_0$ is special, it takes $\mathbf{Y}$ and r as inputs, and it has only one parameter ζ_0 . For simplicity, we use $\Theta = \{\{\zeta_k\}_{k=0}^K, \{\eta_k\}_{k=1}^K\}$ to represent all parameters in this neural network.

Training. Given a training data set $\mathcal{D}_{\text{train}}$ consisting of $(\mathbf{Y}, \mathbf{X}_\star)$ pairs (observation, ground truth), one can train the neural network and obtain parameters Θ by minimizing the following loss function:

$$\underset{\Theta}{\text{minimize}} \mathbb{E}_{(\mathbf{Y}, \mathbf{X}_\star) \sim \mathcal{D}_{\text{train}}} \|\mathbf{L}_K(\mathbf{Y}, \Theta) (\mathbf{R}_K(\mathbf{Y}, \Theta))^\top - \mathbf{X}_\star\|_F^2. \quad (12)$$

Directly minimizing (12) is called end-to-end training, and it can be easily implemented on deep learning platforms nowadays. In this paper, we adopt a more advanced training technique named as *layer-wise training* or *curriculum learning*, which has been proven as a powerful tool on training deep unfolding models [43, 44]. The process of layer-wise training is divided into $K + 1$ stages:

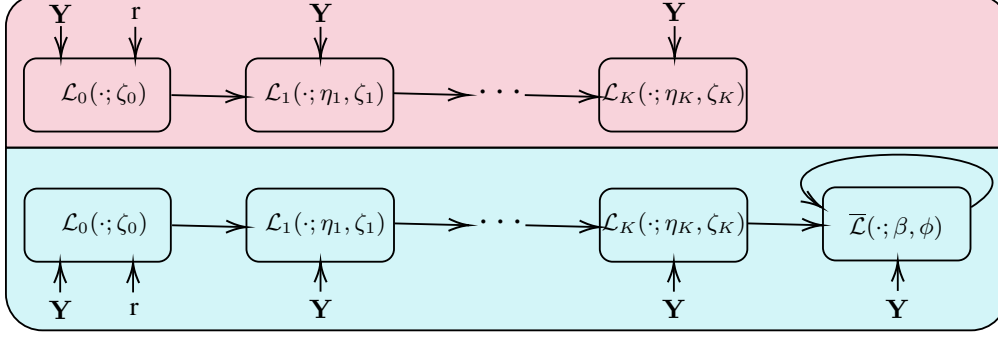


Figure 1: A high-level structure comparison between classic FNN-based deep unfolding (top) and proposed FRMNN-based deep unfolding (bottom). In the diagrams, $\mathcal{L}_k$ denotes the k -th layer of FNN and $\bar{\mathcal{L}}$ is a layer of RNN.

- Training the 0-th layer with $\text{minimize}_{\Theta} \mathbb{E} \|\mathbf{L}_0 \mathbf{R}_0^\top - \mathbf{X}_\star\|_{\text{F}}^2$.
- Training the 1-st layer with $\text{minimize}_{\Theta} \mathbb{E} \|\mathbf{L}_1 \mathbf{R}_1^\top - \mathbf{X}_\star\|_{\text{F}}^2$.
- ...
- Training the final layer with (12): $\text{minimize}_{\Theta} \mathbb{E} \|\mathbf{L}_K \mathbf{R}_K^\top - \mathbf{X}_\star\|_{\text{F}}^2$.

Feedforward-recurrent-mixed neural network. One disadvantage of the aforementioned FNN model is its fixed number of layers. If one wants to go further steps and obtain higher accuracy after training a neural network, one may have to retrain the neural network once more. Recurrent neural network (RNN) has tied parameters over different layers and, consequently, is extendable to infinite layers. However, we find that the starting iterations play significant roles in the convergence (validated in Section 4 later) and their parameters should be trained individually. Thus, we propose a hybrid model that is demonstrated in the bottom part of Figure 1, named as Feedforward-recurrent-mixed neural network (FRMNN). We use a recurrent neural network appended after a K -layer feedforward neural network. When $k \geq K$, in the $(k - K)$ -th loop of the RNN layer, we follow the same calculation procedures with Algorithm 1 and determine the parameters ζ_k and η_k by

$$\eta_k = \beta \eta_{k-1} \quad \text{and} \quad \zeta_k = \phi \zeta_{k-1}. \quad (13)$$

Thus, all the RNN layers share the common parameters β and ϕ .

The training of FRMNN follows in two phases:

- Training the K -layer FNN with layer-wise training.
- Fixing the FNN and searching RNN parameters β and ϕ to minimize the convergence metric at the $(\bar{K} - K)$ -th layer of RNN for some $\bar{K} > K$:

$$\text{minimize}_{\beta, \phi} \mathbb{E}_{(\mathbf{Y}, \mathbf{X}_\star) \sim \mathcal{D}_{\text{train}}} \|\mathbf{L}_{\bar{K}}(\beta, \phi) (\mathbf{R}_{\bar{K}}(\beta, \phi))^\top - \mathbf{X}_\star\|_{\text{F}}^2. \quad (14)$$

Our new model provides the flexibility of training a neural network with finite $\bar{K}$ layers and testing it with infinite layers. Consequently, the stop condition of LRPCA has to be (Run $\bar{K}$ iterations) if the parameters are trained via FNN model; and the stop condition can be (Error < Tolerance) if our FRMNN model is used.

In this paper, we use stochastic gradient descent (SGD) in the layer-wise training stage and use grid search to obtain β and ϕ since there are only two parameters. Moreover, we find that picking $K + 3 \leq \bar{K} \leq K + 5$ is empirically good.

4 Numerical experiments

In this section, we compare the empirical performance of LRPCA with the state-of-the-arts: ScaledGD [12] and AltProj [6]. We hand tune the parameters for ScaledGD and AltProj to achieve their best

performance in the experiments. Moreover, all speed tests are executed on a Windows 10 laptop with Intel i7-8750H CPU, 32GB RAM and Nvidia GTX-1060 GPU. The parameters learning processes are executed on an Ubuntu workstation with Intel i9-9940X CPU, 128GB RAM and two Nvidia RTX-3080 GPUs. All our synthetic test results are averaged over 50 random generated instances, and details of random instance generation can be found in Appendix C. The code for LRPCA is available online at <https://github.com/caesarcai/LRPCA>.

Unfolding models. We compare the performance of LRPCA with the parameters learned from different unfolding models: classic FNN, RNN and proposed FRMNN. In particular, FNN model unrolls 10 iterations of LRPCA and RNN model directly starts the training on the second phase of FRMNN, i.e., $K = 0$. For FRMNN model, we take $K = 10$ and $\bar{K} = 15$ for the training. The test results is summarized in Figure 2. One can see FRMNN model extends FNN model to infinite layers with performance reduction while the pure RNN model drops the convergence performance. Note that the convergence curves of both FRMNN and RNN go down till they reach $\mathcal{O}(10^{-8})$ which is the machine precision since we use single precision in this experiment.

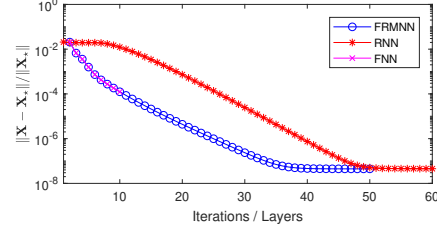


Figure 2: Convergence comparison for FNN-based, RNN-based, and FRMNN-based learning.

Computational efficiency. In this section, we present the speed advantage of LRPCA. First, we compare the convergence behavior of LRPCA to our baseline ScaledGD in Figure 3. We find LRPCA converges in much less iterations, especially when the outlier sparsity α is larger. Next, we compare per-iteration runtime of LRPCA to our baseline ScaledGD in Figure 4. One can see that ScaledGD runs slower when α becomes larger. In contrast, the per-iteration runtime of LRPCA is insensitive to α and is significantly faster than ScaledGD. Finally, we compare the total runtime among LRPCA, ScaledGD and AltProj in Figure 5. Therein, we find LRPCA is substantially faster than the state-of-the-arts if the model has been trained. The training time for models with different sizes and settings are reported in the supplement.

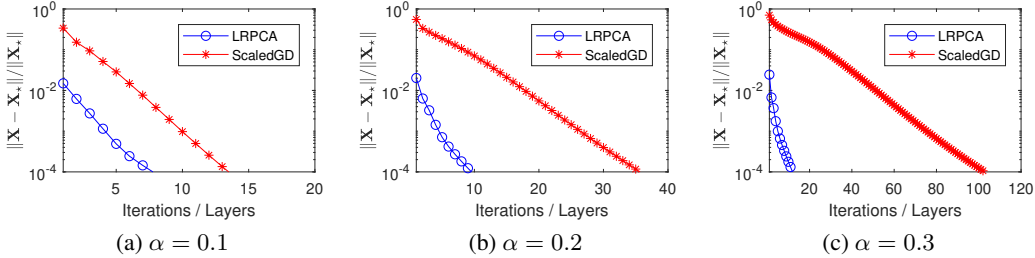


Figure 3: Convergence comparison for LRPCA and baseline ScaledGD with varying outlier sparsity α . Problem dimension $n = 1000$ and rank $r = 5$.

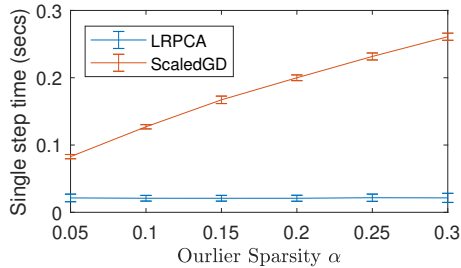


Figure 4: Single-step runtime comparison with error bar for LRPCA and baseline ScaledGD with varying outlier sparsity α . Problem dimension $n = 1000$ and rank $r = 5$. The results are based on running 100 iterations excluding initialization.

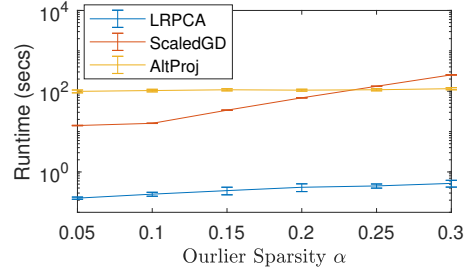


Figure 5: Runtime comparison with error bar for LRPCA and start-of-the-arts (ScaledGD and AltProj) with varying outlier sparsity α . Problem dimension $d = 3000$ and rank $r = 5$. All algorithms halt when $\|Y - X - S\|_F / \|Y\|_F < 10^{-4}$.

Recovery performance. We generate 10 problems for each of the outlier density levels (i.e., α) and compare the recoverabilities of LRPCA and ScaledGD. The test result is presented in the Table 1, where we can conclude that LRPCA is more robust when α is larger.

Table 1: Recovery performance under different outlier density levels.

α	0.4	0.45	0.5	0.55	0.6	0.65	0.7
LRPCA	10/10	10/10	10/10	9/10	8/10	0/10	0/10
ScaledGD	10/10	10/10	0/10	0/10	0/10	0/10	0/10

Video background subtraction. In this section, we apply LRPCA to the task of video background subtraction. We use VIRAT video dataset⁵ as our benchmark, which includes numerous colorful videos with static backgrounds. The videos have been grouped into training and testing sets. We used 65 videos for parameter learning and 4 videos for verification⁶. We first convert all videos to grayscale. Next, each frame of the videos is vectorized and become a matrix column, and all frames of a video form a data matrix. The static backgrounds are the low-rank component of the data matrices and moving objects can be viewed as outliers, thus we can separate the backgrounds and foregrounds via RPCA. The video details and the experimental results are summarized in Table 2 and selected visual results for LRPCA are presented in Figure 6. One can see LRPCA runs much faster than both ScaledGD and AltProj in all verification tests. Furthermore, the background subtraction results of LRPCA are also visually desirable. More details about the experimental setup can be found in Appendix C, along with additional visual results for ScaledGD and AltProj.

Table 2: Video details and runtime comparison for the task of background subtraction. All algorithms halt when $\max(\|\mathbf{X}_k - \mathbf{X}_{k-1}\|_F / \|\mathbf{X}_{k-1}\|_F, \|\mathbf{S}_k - \mathbf{S}_{k-1}\|_F / \|\mathbf{S}_{k-1}\|_F) < 10^{-3}$.

VIDEO NAME	FRAME SIZE	FRAME NUMBER	RUNTIME (secs)		
			LRPCA	ScaledGD	AltProj
ParkingLot1	320×180	965	16.01	260.45	63.04
ParkingLot2	320×180	2149	33.95	639.03	144.50
ParkingLot3	480×270	1110	38.85	662.08	166.91
StreeView	480×270	1034	33.73	626.05	167.66

Ultrasound imaging. We compare LRPCA with CORONA [23], a state-of-the-art learning-based RPCA method, on ultrasound imaging. The testing dataset consists of 2400 training examples and 800 validation examples, which can be downloaded online at <https://www.wisdom.weizmann.ac.il/~yonina>. Each example is of size 1024×40 . The target rank of the low-rank matrix is set to be $r = 1$. Recovery accuracy is measured by the loss function: $\text{loss} = \text{MSE}(X_{\text{output}}, X_{\star})$, where MSE stands for the mean square error. The test results are summarized in Table 3.

Table 3: Test results for ultrasound imaging.

ALGORITHM	AVERAGE INFERENCE TIME	AVERAGE loss
LRPCA	0.0057 secs	9.97×10^{-4}
CORONA	0.9225 secs	4.88×10^{-4}

Note that LRPCA is designed for generic RPCA and CORONA is specifically designed for ultrasound imaging. Thus, it is not a surprise that our recovery accuracy is slightly worse than CORONA.

⁵Available at: <https://viratdata.org>.

⁶The names of the tested videos are corresponding to these original video numbers in VIRAT dataset: ParkingLot1: S_000204_06_001527_001560, ParkingLot2: S_010100_05_000917_001017, ParkingLot3: S_050204_00_000000_000066, StreeView: S_050100_13_002202_002244.

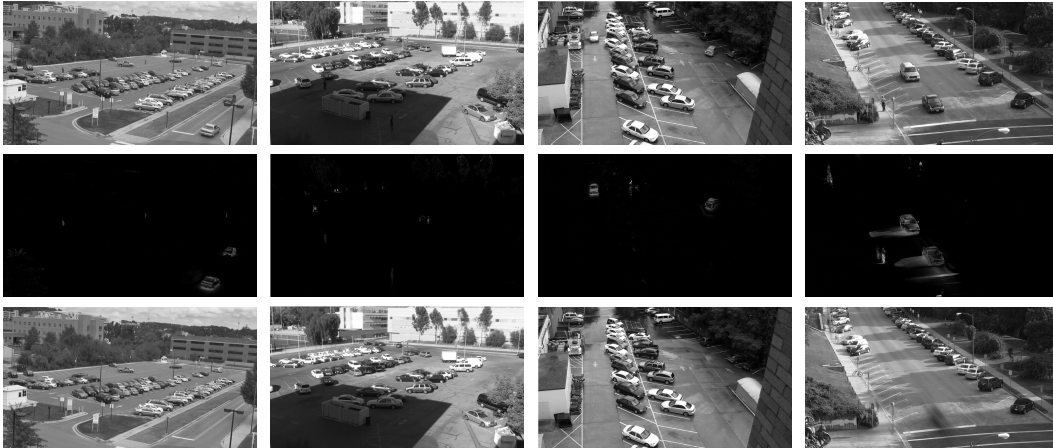


Figure 6: Video background subtraction visual results for LRPCA. Each column represents a selected frame from the tested videos. From left to right, they are ParkingLot1, ParkingLot2, ParkingLot3, and StreetView. The first row contains the original frames. The next two rows are the separated foreground and background produced by LRPCA, respectively. More visual results for ScaledGD and AltProj are available in Appendix C.

However, the average runtime of LRPCA is substantially faster than CORONA. We believe that our speed advantage will be even more significant on larger-scale examples, which is indeed our main contribution: scalability.

More numerical experiments. There are some other interesting empirical problems worth to explore and we put the extra experimental results in Appendix C due to the space limitation. They are summarized here: 1. Our model has good generalization ability. We can train our model on small-size and low-rank training instances (say, $n = 1000$ and $r = 5$) and use the model DIRECTLY on problems with larger size (say, $n = 3000$ and $r = 5$) or with higher rank (say, $n = 1000$ and $r = 15$) with good performance. 2. The learned parameters are explainable. With the learned parameters, LRPCA goes very aggressively with large step sizes in the first several steps and tends to be conservative after that.

5 Conclusion and future work

We have presented a highly efficient learning-based approach for high-dimensional RPCA problems. In addition, we have presented a novel feedforward-recurrent-mixed neural network model which can learn the parameters for potentially infinite iterations without performance reduction. Theoretical recovery guarantee has been established for the proposed algorithm. The numerical experiments show that the proposed approach is superior to the state-of-the-arts.

One of our future directions is to study learning-based stochastic approach. While this paper focuses on deterministic RPCA approaches, the stochastic RPCA approaches, e.g., partialGD [8], PG-RMC [9], and IRCUR [11], have shown promising speed advantage, especially for large-scale problems. Another future direction is to explore robust tensor decomposition incorporate with deep learning, as some preliminary studies have shown the advantages of tensor structure in certain machine learning tasks [45, 46].

Broader impact

RPCA has been broadly applied as one of fundamental techniques in data mining and machine learning communities. Hence, through its applications, the proposed learning-based approach for RPCA has potential broader impacts that data mining and machine learning have, as well as their limitations.

References

- [1] John Wright, Arvind Ganesh, Shankar Rao, Yigang Peng, and Yi Ma. Robust principal component analysis: Exact recovery of corrupted low-rank matrices via convex optimization. In *Advances in Neural Information Processing Systems*, pages 2080–2088, 2009.
- [2] Zhouchen Lin, Minming Chen, and Yi Ma. The augmented Lagrange multiplier method for exact recovery of corrupted low-rank matrices. *arXiv preprint arXiv:1009.5055*, 2010.
- [3] Emmanuel J Candès, Xiaodong Li, Yi Ma, and John Wright. Robust principal component analysis? *Journal of the ACM*, 58(3):11, 2011.
- [4] Daniel Hsu, Sham M Kakade, and Tong Zhang. Robust matrix decomposition with sparse corruptions. *IEEE Transactions on Information Theory*, 57(11):7221–7234, 2011.
- [5] Yudong Chen, Ali Jalali, Sujay Sanghavi, and Constantine Caramanis. Low-rank matrix recovery from errors and erasures. *IEEE Transactions on Information Theory*, 59(7):4324–4337, 2013.
- [6] Praneeth Netrapalli, UN Niranjan, Sujay Sanghavi, Animashree Anandkumar, and Prateek Jain. Non-convex robust PCA. In *Advances in Neural Information Processing Systems*, pages 1107–1115, 2014.
- [7] Quanquan Gu, Zhaoran Wang, and Han Liu. Low-rank and sparse structure pursuit via alternating minimization. In *Artificial Intelligence and Statistics*, pages 600–609, 2016.
- [8] Xinyang Yi, Dohyung Park, Yudong Chen, and Constantine Caramanis. Fast algorithms for robust PCA via gradient descent. In *Advances in Neural Information Processing Systems*, pages 4152–4160, 2016.
- [9] Yeshwanth Cherapanamjeri, Kartik Gupta, and Prateek Jain. Nearly optimal robust matrix completion. In *International Conference on Machine Learning*, pages 797–805, 2017.
- [10] HanQin Cai, Jian-Feng Cai, and Ke Wei. Accelerated alternating projections for robust principal component analysis. *The Journal of Machine Learning Research*, 20(1):685–717, 2019.
- [11] HanQin Cai, Keaton Hamm, Longxiu Huang, Jiaqi Li, and Tao Wang. Rapid robust principal component analysis: CUR accelerated inexact low rank estimation. *IEEE Signal Processing Letters*, 28:116–120, 2020.
- [12] Tian Tong, Cong Ma, and Yuejie Chi. Accelerating ill-conditioned low-rank matrix estimation via scaled gradient descent. *The Journal of Machine Learning Research*, 22(150):1–63, 2021.
- [13] HanQin Cai, Keaton Hamm, Longxiu Huang, and Deanna Needell. Robust CUR decomposition: Theory and imaging applications. *SIAM Journal on Imaging Sciences*, 14(4):1472–1503, 2021.
- [14] Won-Dong Jang, Chulwoo Lee, and Chang-Su Kim. Primary object segmentation in videos via alternate convex optimization of foreground and background distributions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 696–704, 2016.
- [15] Po-Sen Huang, Scott Deeann Chen, Paris Smaragdis, and Mark Hasegawa-Johnson. Singing-voice separation from monaural recordings using robust principal component analysis. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 57–60, 2012.
- [16] John Wright, Allen Y Yang, Arvind Ganesh, S Shankar Sastry, and Yi Ma. Robust face recognition via sparse representation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 31(2):210–227, 2008.
- [17] Yigang Peng, Arvind Ganesh, John Wright, Wenli Xu, and Yi Ma. RASL: Robust alignment by sparse and low-rank decomposition for linearly correlated images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 34(11):2233–2246, 2012.
- [18] Jin-Xing Liu, Yong Xu, Chun-Hou Zheng, Heng Kong, and Zhi-Hui Lai. RPCA-based tumor classification using gene expression data. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 12(4):964–970, 2014.

- [19] Yudong Chen, Sujay Sanghavi, and Huan Xu. Clustering sparse graphs. In *Advances in Neural Information Processing Systems*, pages 2204–2212, 2012.
- [20] Yvon Tharrault, Gilles Mourot, José Ragot, and Didier Maquin. Fault detection and isolation with robust principal component analysis. *International Journal of Applied Mathematics and Computer Science*, 18(4):429–442, 2008.
- [21] HanQin Cai, Jian-Feng Cai, Tianming Wang, and Guojian Yin. Accelerated structured alternating projections for robust spectrally sparse signal recovery. *IEEE Transactions on Signal Processing*, 69:809–821, 2021.
- [22] Karol Gregor and Yann LeCun. Learning fast approximations of sparse coding. In *Proceedings of the 27th International Conference on International Conference on Machine Learning*, pages 399–406, 2010.
- [23] Regev Cohen, Yi Zhang, Oren Solomon, Daniel Toberman, Liran Taieb, Ruud JG van Sloun, and Yonina C Eldar. Deep convolutional robust PCA with application to ultrasound imaging. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 3212–3216, 2019.
- [24] Oren Solomon, Regev Cohen, Yi Zhang, Yi Yang, Qiong He, Jianwen Luo, Ruud JG van Sloun, and Yonina C Eldar. Deep unfolded robust PCA with application to clutter suppression in ultrasound. *IEEE Transactions on Medical Imaging*, 39(4):1051–1063, 2019.
- [25] Huynh Van Luong, Boris Joukovsky, Yonina C Eldar, and Nikos Deligiannis. A deep-unfolded reference-based RPCA network for video foreground-background separation. In *European Signal Processing Conference*, pages 1432–1436, 2021.
- [26] Jian-Feng Cai, Emmanuel J Candès, and Zuowei Shen. A singular value thresholding algorithm for matrix completion. *SIAM Journal on Optimization*, 20(4):1956–1982, 2010.
- [27] Søren Hauberg, Aasa Feragen, Raffi Enficiaud, and Michael J Black. Scalable robust principal component analysis using Grassmann averages. *IEEE transactions on pattern analysis and machine intelligence*, 38(11):2298–2311, 2015.
- [28] Rudrasis Chakraborty, Liu Yang, Soren Hauberg, and Baba Vemuri. Intrinsic Grassmann averages for online linear, robust and nonlinear subspace learning. *IEEE transactions on pattern analysis and machine intelligence*, 2020.
- [29] Bo Xin, Yizhou Wang, Wen Gao, David Wipf, and Baoyuan Wang. Maximal sparsity with deep networks? *Advances in Neural Information Processing Systems*, 29:4340–4348, 2016.
- [30] Yan Yang, Jian Sun, Huibin Li, and Zongben Xu. Deep ADMM-Net for compressive sensing MRI. In *Proceedings of the 30th International Conference on Neural Information Processing Systems*, pages 10–18, 2016.
- [31] Christopher A Metzler, Ali Mousavi, and Richard G Baraniuk. Learned D-AMP: Principled neural network based compressive image recovery. *Advances in Neural Information Processing Systems*, pages 1773–1784, 2017.
- [32] Jian Zhang and Bernard Ghanem. ISTA-Net: Interpretable optimization-inspired deep network for image compressive sensing. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1828–1837, 2018.
- [33] Jonas Adler and Ozan Öktem. Learned primal-dual reconstruction. *IEEE Transactions on Medical Imaging*, 37(6):1322–1332, 2018.
- [34] Jialin Liu, Xiaohan Chen, Zhangyang Wang, and Wotao Yin. ALISTA: Analytic weights are as good as learned weights in LISTA. In *International Conference on Learning Representations*, 2019.
- [35] Jiayi Shen, Xiaohan Chen, Howard Heaton, Tianlong Chen, Jialin Liu, Wotao Yin, and Zhangyang Wang. Learning a minimax optimizer: A pilot study. In *International Conference on Learning Representations*, 2020.

- [36] Vishal Monga, Yuelong Li, and Yonina C Eldar. Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing. *IEEE Signal Processing Magazine*, 38(2):18–44, 2021.
- [37] Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando De Freitas. Learning to learn by gradient descent by gradient descent. In *Advances in Neural Information Processing Systems*, pages 3981–3989, 2016.
- [38] Olga Wichrowska, Niru Maheswaranathan, Matthew W Hoffman, Sergio Gomez Colmenarejo, Misha Denil, Nando Freitas, and Jascha Sohl-Dickstein. Learned optimizers that scale and generalize. In *International Conference on Machine Learning*, pages 3751–3760. PMLR, 2017.
- [39] Luke Metz, Niru Maheswaranathan, Jeremy Nixon, Daniel Freeman, and Jascha Sohl-Dickstein. Understanding and correcting pathologies in the training of learned optimizers. In *International Conference on Machine Learning*, pages 4556–4565. PMLR, 2019.
- [40] Calypso Herrera, Florian Krach, Anastasis Kratsios, Pierre Ruysen, and Josef Teichmann. Denise: Deep learning based robust PCA for positive semidefinite matrices. *arXiv preprint arXiv:2004.13612*, 2020.
- [41] Ivan Dokmanic, Reza Parhizkar, Juri Ranieri, and Martin Vetterli. Euclidean distance matrices: essential theory, algorithms, and applications. *IEEE Signal Processing Magazine*, 32(6):12–30, 2015.
- [42] Venkat Chandrasekaran, Sujay Sanghavi, Pablo A Parrilo, and Alan S Willsky. Rank-sparsity incoherence for matrix decomposition. *SIAM Journal on Optimization*, 21(2):572–596, 2011.
- [43] Xiaohan Chen, Jialin Liu, Zhangyang Wang, and Wotao Yin. Theoretical linear convergence of unfolded ISTA and its practical weights and thresholds. In *Advances in Neural Information Processing Systems*, pages 9079–9089, 2018.
- [44] Tianlong Chen, Weiyi Zhang, Zhou Jingyang, Shiyu Chang, Sijia Liu, Lisa Amini, and Zhangyang Wang. Training stronger baselines for learning to optimize. In *Advances in Neural Information Processing Systems*, pages 7332–7343, 2020.
- [45] HanQin Cai, Zehan Chao, Longxiu Huang, and Deanna Needell. Fast robust tensor principal component analysis via fiber CUR decomposition. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*, pages 189–197, 2021.
- [46] Jian-Feng Cai, Jingyang Li, and Dong Xia. Generalized low-rank plus sparse tensor estimation by fast Riemannian optimization. *arXiv preprint arXiv:2103.08895*, 2021.

Supplementary Material for Learned Robust PCA: A Scalable Deep Unfolding Approach for High-Dimensional Outlier Detection

A Proofs

In this section, we provide the mathematical proofs for the claimed theoretical results. Note that the proof of our convergence theorem follows the route established in [12]. However, the details of our proof are quite involved since we replaced the sparsification operator which substantially changes the method of outlier detection.

Let $\mathbf{L}_\star := \mathbf{U}_\star \Sigma_\star^{1/2}$ and $\mathbf{R}_\star := \mathbf{V}_\star \Sigma_\star^{1/2}$ where $\mathbf{U}_\star \Sigma_\star \mathbf{V}_\star^\top$ is the compact SVD of $\mathbf{X}_\star$. For theoretical analysis, we consider the error metric for decomposed rank- r matrices:

$$\text{dist}(\mathbf{L}, \mathbf{R}; \mathbf{L}_\star, \mathbf{R}_\star) := \inf_{\mathbf{Q} \in \mathbb{R}^{r \times r}, \text{rank}(\mathbf{Q})=r} \left(\|(\mathbf{L}\mathbf{Q} - \mathbf{L}_\star) \Sigma_\star^{1/2}\|_F^2 + \|(\mathbf{R}\mathbf{Q}^{-\top} - \mathbf{R}_\star) \Sigma_\star^{1/2}\|_F^2 \right)^{1/2}.$$

Notice that the optimal alignment matrix $\mathbf{Q}$ exists and invertible if $\mathbf{L}$ and $\mathbf{R}$ are sufficiently close to $\mathbf{L}_\star$ and $\mathbf{R}_\star$. In particular, one can have the following lemma.

Lemma 2 ([12, Lemma 9]). *For any $\mathbf{L} \in \mathbb{R}^{n_1 \times r}$ and $\mathbf{R} \in \mathbb{R}^{n_2 \times r}$, if*

$$\text{dist}(\mathbf{L}, \mathbf{R}; \mathbf{L}_\star, \mathbf{R}_\star) < c\sigma_r(\mathbf{X}_\star)$$

for some $0 < c < 1$, then the optimal alignment matrix $\mathbf{Q}$ between $[\mathbf{L}, \mathbf{R}]$ and $[\mathbf{L}_\star, \mathbf{R}_\star]$ exists and is invertible.

More notation. In addition to the notation introduced in Section 1, we provide some more notation for the analysis. $\vee$ denotes the logical disjunction, which takes the max of two operands. For any matrix $\mathbf{M}$, $\|\mathbf{M}\|_2 = \sigma_1(\mathbf{M})$ denotes the spectral norm and $\|\mathbf{M}\|_{1,\infty} = \max_i \sum_j |\mathbf{M}_{i,j}|$ denotes the largest row-wise ℓ_1 norm.

For ease of presentation, we take $n := n_1 = n_2$ in the rest of this section, but we emphasize that similar results can be easily drawn for the rectangular matrix setting. Furthermore, we introduce following shorthand for notational convenience: $\mathbf{Q}_k$ denotes the optimal alignment matrix between $(\mathbf{L}_k, \mathbf{R}_k)$ and $(\mathbf{L}_\star, \mathbf{R}_\star)$, $\mathbf{L}_{\natural} := \mathbf{L}_k \mathbf{Q}_k$, $\mathbf{R}_{\natural} := \mathbf{R}_k \mathbf{Q}_k^{-\top}$, $\Delta_L := \mathbf{L}_{\natural} - \mathbf{L}_\star$, $\Delta_R := \mathbf{R}_{\natural} - \mathbf{R}_\star$, and $\Delta_S := \mathbf{S}_{k+1} - \mathbf{S}_\star$.

A.1 Proof of Theorem 1

We first present the theorems of local linear convergence and guaranteed initialization. The proofs of these two theorems can be find in Sections A.3 and A.4, respectively.

Theorem 3 (Local linear convergence). *Suppose that $\mathbf{X}_\star = \mathbf{L}_\star \mathbf{R}_\star^\top$ is a rank- r matrix with μ -incoherence and $\mathbf{S}_\star$ is an α -sparse matrix with $\alpha \leq \frac{1}{10^4 \mu r^{1.5}}$. Let $\mathbf{Q}_k$ be the optimal alignment matrix between $[\mathbf{L}_k, \mathbf{R}_k]$ and $[\mathbf{L}_\star, \mathbf{R}_\star]$. If the initial guesses obey the conditions*

$$\text{dist}(\mathbf{L}_0, \mathbf{R}_0; \mathbf{L}_\star, \mathbf{R}_\star) \leq \varepsilon_0 \sigma_r(\mathbf{X}_\star),$$

$$\|(\mathbf{L}_0 \mathbf{Q}_0 - \mathbf{L}_\star) \Sigma_\star^{1/2}\|_{2,\infty} \vee \|(\mathbf{R}_0 \mathbf{Q}_0^{-\top} - \mathbf{R}_\star) \Sigma_\star^{1/2}\|_{2,\infty} \leq \sqrt{\frac{\mu r}{n}} \sigma_r(\mathbf{X}_\star)$$

with $\varepsilon_0 := 0.02$, then by setting the thresholding values $\zeta_k = \|\mathbf{X}_\star - \mathbf{L}_{k-1} \mathbf{R}_{k-1}^\top\|_\infty$ and the fixed step size $\eta_k = \eta \in [\frac{1}{4}, \frac{8}{9}]$, the iterates of Algorithm 1 satisfy

$$\text{dist}(\mathbf{L}_k, \mathbf{R}_k; \mathbf{L}_\star, \mathbf{R}_\star) \leq \varepsilon_0 \tau^k \sigma_r(\mathbf{X}_\star),$$

$$\|(\mathbf{L}_k \mathbf{Q}_k - \mathbf{L}_\star) \Sigma_\star^{1/2}\|_{2,\infty} \vee \|(\mathbf{R}_k \mathbf{Q}_k^{-\top} - \mathbf{R}_\star) \Sigma_\star^{1/2}\|_{2,\infty} \leq \sqrt{\frac{\mu r}{n}} \tau^k \sigma_r(\mathbf{X}_\star),$$

where the convergence rate $\tau := 1 - 0.6\eta$.

Theorem 4 (Guaranteed initialization). Suppose that $\mathbf{X}_\star = \mathbf{L}_\star \mathbf{R}_\star^\top$ is a rank- r matrix with μ -incoherence and $\mathbf{S}_\star$ is an α -sparse matrix with $\alpha \leq \frac{c_0}{\mu r^{1.5\kappa}}$ for some small positive constant $c_0 \leq \frac{1}{35}$. Let $\mathbf{Q}_0$ be the optimal alignment matrix between $[\mathbf{L}_0, \mathbf{R}_0]$ and $[\mathbf{L}_\star, \mathbf{R}_\star]$. By setting the thresholding values $\zeta_0 = \|\mathbf{X}_\star\|_\infty$, the initial guesses satisfy

$$\begin{aligned} \text{dist}(\mathbf{L}_0, \mathbf{R}_0; \mathbf{L}_\star, \mathbf{R}_\star) &\leq 10c_0\sigma_r(\mathbf{X}_\star), \\ \|(\mathbf{L}_0\mathbf{Q}_0 - \mathbf{L}_\star)\Sigma_\star^{1/2}\|_{2,\infty} \vee \|(\mathbf{R}_0\mathbf{Q}_0^{-\top} - \mathbf{R}_\star)\Sigma_\star^{1/2}\|_{2,\infty} &\leq \sqrt{\frac{\mu r}{n}}\sigma_r(\mathbf{X}_\star). \end{aligned}$$

In addition, we present a lemma that verifies our selection of thresholding values is indeed effective.

Lemma 5. At the $(k+1)$ -th iteration of Algorithm 1, taking the thresholding value $\zeta_{k+1} := \|\mathbf{X}_\star - \mathbf{X}_k\|_\infty$ gives

$$\|\mathbf{S}_\star - \mathbf{S}_{k+1}\|_\infty \leq 2\|\mathbf{X}_\star - \mathbf{X}_k\|_\infty \quad \text{and} \quad \text{supp}(\mathbf{S}_{k+1}) \subseteq \text{supp}(\mathbf{S}_\star).$$

Proof. Denote $\Omega_\star := \text{supp}(\mathbf{S}_\star)$ and $\Omega_{k+1} := \text{supp}(\mathbf{S}_{k+1})$. Recall that $\mathbf{S}_{k+1} = \mathcal{S}_{\zeta_{k+1}}(\mathbf{Y} - \mathbf{X}_k) = \mathcal{S}_{\zeta_{k+1}}(\mathbf{S}_\star + \mathbf{X}_\star - \mathbf{X}_k)$. Since $[\mathbf{S}_\star]_{i,j} = 0$ outside its support, so $[\mathbf{Y} - \mathbf{X}_k]_{i,j} = [\mathbf{X}_\star - \mathbf{X}_k]_{i,j}$ for the entries $(i, j) \in \Omega_\star^c$. Applying the chosen thresholding value $\zeta_{k+1} := \|\mathbf{X}_\star - \mathbf{X}_k\|_\infty$, one have $[\mathbf{S}_{k+1}]_{i,j} = 0$ for all $(i, j) \in \Omega_\star^c$. Hence, the support of $\mathbf{S}_{k+1}$ must belongs to the support of $\mathbf{S}_\star$, i.e.,

$$\text{supp}(\mathbf{S}_{k+1}) = \Omega_{k+1} \subseteq \Omega_\star = \text{supp}(\mathbf{S}_\star).$$

This proves our first claim.

Obviously, $[\mathbf{S}_\star - \mathbf{S}_{k+1}]_{i,j} = 0$ for all $(i, j) \in \Omega_\star^c$. Moreover, we can split the entries in $\Omega_\star$ into two groups:

$$\begin{aligned} \Omega_{k+1} &= \{(i, j) \mid |[\mathbf{Y} - \mathbf{X}_k]_{i,j}| > \zeta_{k+1} \text{ and } [\mathbf{S}_\star]_{i,j} \neq 0\} \quad \text{and} \\ \Omega_\star \setminus \Omega_{k+1} &= \{(i, j) \mid |[\mathbf{Y} - \mathbf{X}_k]_{i,j}| \leq \zeta_{k+1} \text{ and } [\mathbf{S}_\star]_{i,j} \neq 0\}, \end{aligned}$$

and it holds

$$\begin{aligned} |[\mathbf{S}_\star - \mathbf{S}_{k+1}]_{i,j}| &= \left| \frac{[\mathbf{X}_k - \mathbf{X}_\star]_{i,j} - \text{sign}([\mathbf{Y} - \mathbf{X}_k]_{i,j})\zeta_{k+1}}{[\mathbf{S}_\star]_{i,j}} \right| \\ &\leq \begin{cases} |[\mathbf{X}_k - \mathbf{X}_\star]_{i,j}| + \zeta_{k+1} \\ |[\mathbf{X}_\star - \mathbf{X}_k]_{i,j}| + \zeta_{k+1} \end{cases} \\ &\leq \begin{cases} 2\|\mathbf{X}_\star - \mathbf{X}_k\|_\infty & (i, j) \in \Omega_{k+1}, \\ 2\|\mathbf{X}_\star - \mathbf{X}_k\|_\infty & (i, j) \in \Omega_\star \setminus \Omega_{k+1}. \end{cases} \end{aligned}$$

Therefore, we can conclude $\|\mathbf{S}_\star - \mathbf{S}_{k+1}\|_\infty \leq 2\|\mathbf{X}_\star - \mathbf{X}_k\|_\infty$. $\square$

Now, we are already to prove Theorem 1.

Proof of Theorem 1. Take $c_0 = 10^{-4}$ in Theorem 4. Thus, the results of Theorem 4 satisfy the condition of Theorem 3, and gives

$$\begin{aligned} \text{dist}(\mathbf{L}_k, \mathbf{R}_k; \mathbf{L}_\star, \mathbf{R}_\star) &\leq 0.02(1 - 0.6\eta)^k\sigma_r(\mathbf{X}_\star), \\ \|(\mathbf{L}_k\mathbf{Q}_k - \mathbf{L}_\star)\Sigma_\star^{1/2}\|_{2,\infty} \vee \|(\mathbf{R}_k\mathbf{Q}_k^{-\top} - \mathbf{R}_\star)\Sigma_\star^{1/2}\|_{2,\infty} &\leq \sqrt{\frac{\mu r}{n}}(1 - 0.6\eta)^k\sigma_r(\mathbf{X}_\star) \end{aligned}$$

for all $k \geq 0$. [12, Lemma 3] states that

$$\|\mathbf{L}_k\mathbf{R}_k^\top - \mathbf{X}_\star\|_F \leq 1.5 \text{dist}(\mathbf{L}_k, \mathbf{R}_k; \mathbf{L}_\star, \mathbf{R}_\star)$$

as long as $\|(\mathbf{L}_k\mathbf{Q}_k - \mathbf{L}_\star)\Sigma_\star^{1/2}\|_{2,\infty} \vee \|(\mathbf{R}_k\mathbf{Q}_k^{-\top} - \mathbf{R}_\star)\Sigma_\star^{1/2}\|_{2,\infty} \leq \sqrt{\frac{\mu r}{n}}\sigma_r(\mathbf{X}_\star)$. Hence, our first claim is proved.

When $k \geq 1$, the second claim is directly followed by Lemma 5. When $k = 0$, take $\mathbf{X}_{-1} = \mathbf{0}$, then one can see $\mathbf{S}_0 = \mathcal{S}_{\zeta_0}(\mathbf{Y}) = \mathcal{S}_{\zeta_0}(\mathbf{Y} - \mathbf{X}_{-1})$ where $\zeta_0 = \|\mathbf{X}_\star\|_\infty = \|\mathbf{X}_\star - \mathbf{X}_{-1}\|_\infty$. Applying Lemma 5 again, we have the second claim for all $k \geq 0$.

This finishes the proof. $\square$

A.2 Auxiliary lemmata

Before we can present the proofs for Theorems 3 and 4, several important auxiliary lemmata must be processed.

Lemma 6. *For any α -sparse matrix $\mathbf{S} \in \mathbb{R}^{n \times n}$, the following inequalities hold:*

$$\begin{aligned}\|\mathbf{S}\|_2 &\leq \alpha n \|\mathbf{S}\|_\infty, \\ \|\mathbf{S}\|_{2,\infty} &\leq \sqrt{\alpha n} \|\mathbf{S}\|_\infty, \\ \|\mathbf{S}\|_{1,\infty} &\leq \alpha n \|\mathbf{S}\|_\infty.\end{aligned}$$

Proof. The first claim has been shown as [6, Lemma 4]. The rest two claims are directly followed by the fact $\mathbf{S}$ has at most αn non-zero elements in each row and each column. $\square$

Lemma 7. *If*

$$\text{dist}(\mathbf{L}_k, \mathbf{R}_k; \mathbf{L}_\star, \mathbf{R}_\star) \leq \varepsilon_0 \tau^k \sigma_r(\mathbf{X}_\star),$$

then the following inequalities hold

$$\begin{aligned}\|\Delta_L \Sigma_\star^{1/2}\|_F \vee \|\Delta_R \Sigma_\star^{1/2}\|_F &\leq \varepsilon_0 \tau^k \sigma_r(\mathbf{X}_\star) \\ \|\Delta_L \Sigma_\star^{1/2}\|_2 \vee \|\Delta_R \Sigma_\star^{1/2}\|_2 &\leq \varepsilon_0 \tau^k \sigma_r(\mathbf{X}_\star)\end{aligned}$$

Proof. Recall that $\text{dist}(\mathbf{L}_k, \mathbf{R}_k; \mathbf{L}_\star, \mathbf{R}_\star) = \sqrt{\|\Delta_L \Sigma_\star^{1/2}\|_F^2 \vee \|\Delta_R \Sigma_\star^{1/2}\|_F^2}$. The first claim is directly followed by the definition of dist .

By the fact that $\|\mathbf{A}\|_2 \leq \|\mathbf{A}\|_F$ for any matrix, we deduct the second claim from the first claim. $\square$

Lemma 8. *If*

$$\text{dist}(\mathbf{L}_k, \mathbf{R}_k; \mathbf{L}_\star, \mathbf{R}_\star) \leq \varepsilon_0 \tau^k \sigma_r(\mathbf{X}_\star),$$

then it holds

$$\|\mathbf{L}_k(\mathbf{L}_k^\top \mathbf{L}_k)^{-1} \Sigma_\star^{1/2}\|_2 \vee \|\mathbf{R}_k(\mathbf{R}_k^\top \mathbf{R}_k)^{-1} \Sigma_\star^{1/2}\|_2 \leq \frac{1}{1 - \varepsilon_0}.$$

Proof. [12, Lemma 12] provides the following inequalities:

$$\begin{aligned}\|\mathbf{L}_k(\mathbf{L}_k^\top \mathbf{L}_k)^{-1} \Sigma_\star^{1/2}\|_2 &\leq \frac{1}{1 - \|\Delta_L \Sigma_\star^{-1/2}\|_2}, \\ \|\mathbf{R}_k(\mathbf{R}_k^\top \mathbf{R}_k)^{-1} \Sigma_\star^{1/2}\|_2 &\leq \frac{1}{1 - \|\Delta_R \Sigma_\star^{-1/2}\|_2},\end{aligned}$$

as long as $\|\Delta_L \Sigma_\star^{-1/2}\|_2 \vee \|\Delta_R \Sigma_\star^{-1/2}\|_2 < 1$.

By Lemma 7, we have $\|\Delta_L \Sigma_\star^{-1/2}\|_2 \vee \|\Delta_R \Sigma_\star^{-1/2}\|_2 \leq \varepsilon_0 \tau^k \leq \varepsilon_0$, given $\tau = 1 - 0.6\eta < 1$. The proof is finished since $\varepsilon_0 = 0.02 < 1$. $\square$

Lemma 9. *If*

$$\|(\mathbf{L}_{k+1} \mathbf{Q}_k - \mathbf{L}_\star) \Sigma_\star^{1/2}\|_2 \vee \|(\mathbf{R}_{k+1} \mathbf{Q}_k^\top - \mathbf{R}_\star) \Sigma_\star^{1/2}\|_2 \leq \varepsilon_0 \tau^{k+1} \sigma_r(\mathbf{X}_\star),$$

then

$$\|\Sigma_\star^{1/2} \mathbf{Q}_k^{-1} (\mathbf{Q}_{k+1} - \mathbf{Q}_k) \Sigma_\star^{1/2}\|_2 \vee \|\Sigma_\star^{1/2} \mathbf{Q}_k^\top (\mathbf{Q}_{k+1} - \mathbf{Q}_k)^{-\top} \Sigma_\star^{1/2}\|_2 \leq \frac{2\varepsilon_0}{1 - \varepsilon_0} \sigma_r(\mathbf{X}_\star).$$

Proof. [12, Lemma 14] provides the inequalities:

$$\|\Sigma_\star^{1/2} \tilde{\mathbf{Q}}^{-1} \hat{\mathbf{Q}} \Sigma_\star^{1/2} - \Sigma_\star\|_2 \leq \frac{\|\mathbf{R}(\tilde{\mathbf{Q}}^{-\top} - \hat{\mathbf{Q}}^{-\top}) \Sigma_\star^{1/2}\|_2}{1 - \|(\mathbf{R} \hat{\mathbf{Q}}^{-\top} - \mathbf{R}_\star) \Sigma_\star^{-1/2}\|_2}$$

$$\|\Sigma_\star^{1/2} \tilde{Q}^\top \hat{Q}^{-\top} \Sigma_\star^{1/2} - \Sigma_\star\|_2 \leq \frac{\|L(\tilde{Q} - \hat{Q})\Sigma_\star^{1/2}\|_2}{1 - \|(L\hat{Q} - L_\star)\Sigma_\star^{-1/2}\|_2}$$

for any $L, R \in \mathbb{R}^{n \times r}$ and invertible $\tilde{Q}, \hat{Q} \in \mathbb{R}^{r \times r}$, as long as $\|(L\hat{Q} - L_\star)\Sigma_\star^{-1/2}\|_2 \vee \|(R\hat{Q}^{-\top} - L_\star)\Sigma_\star^{-1/2}\|_2 < 1$.

We will focus on the first term for now. By the assumption of this lemma and the definition of Q_{k+1} , we have

$$\begin{aligned} \|(\mathbf{R}_{k+1} Q_k^{-\top} - \mathbf{R}_\star)\Sigma_\star^{1/2}\|_2 &\leq \varepsilon_0 \tau^{k+1} \sigma_r(\mathbf{X}_\star), \\ \|(\mathbf{R}_{k+1} Q_{k+1}^{-\top} - \mathbf{R}_\star)\Sigma_\star^{1/2}\|_2 &\leq \varepsilon_0 \tau^{k+1} \sigma_r(\mathbf{X}_\star), \\ \|(\mathbf{R}_{k+1} Q_{k+1}^{-\top} - \mathbf{R}_\star)\Sigma_\star^{-1/2}\|_2 &\leq \varepsilon_0 \tau^{k+1}. \end{aligned}$$

Thus, by taking $R = \mathbf{R}_{k+1}$, $\tilde{Q} = Q_k$, and $\hat{Q} = Q_{k+1}$, we obtain

$$\begin{aligned} \|\Sigma_\star^{1/2} Q_k^{-1} (Q_{k+1} - Q_k) \Sigma_\star^{1/2}\|_2 &= \|\Sigma_\star^{1/2} Q_k^{-1} Q_{k+1} \Sigma_\star^{1/2} - \Sigma_\star\|_2 \\ &\leq \frac{\|\mathbf{R}_{k+1} (Q_k^{-\top} - Q_{k+1}^{-\top}) \Sigma_\star^{1/2}\|_2}{1 - \|(\mathbf{R}_{k+1} Q_{k+1}^{-\top} - \mathbf{R}_\star) \Sigma_\star^{-1/2}\|_2} \\ &\leq \frac{\|(\mathbf{R}_{k+1} Q_k^{-\top} - \mathbf{R}_\star) \Sigma_\star^{1/2}\|_2 + \|(\mathbf{R}_{k+1} Q_{k+1}^{-\top} - \mathbf{R}_\star) \Sigma_\star^{1/2}\|_2}{1 - \|(\mathbf{R}_{k+1} Q_{k+1}^{-\top} - \mathbf{R}_\star) \Sigma_\star^{-1/2}\|_2} \\ &\leq \frac{2\varepsilon_0 \tau^{k+1}}{1 - \varepsilon_0 \tau^{k+1}} \sigma_r(\mathbf{X}_\star) \\ &\leq \frac{2\varepsilon_0}{1 - \varepsilon_0} \sigma_r(\mathbf{X}_\star), \end{aligned}$$

provided $\tau = 1 - 0.6\eta < 1$. Similarly, one can see

$$\|\Sigma_\star^{1/2} Q_k^\top (Q_{k+1} - Q_k)^{-\top} \Sigma_\star^{1/2}\|_2 \leq \frac{2\varepsilon_0}{1 - \varepsilon_0} \sigma_r(\mathbf{X}_\star).$$

This finishes the proof. $\square$

Notice that Lemma 9 will be only be used in the proof of Lemma 12. In the meantime, the assumption of Lemma 9 is verified in (16) (see the proof of Lemma 11).

Lemma 10. *If*

$$\begin{aligned} \text{dist}(\mathbf{L}_k, \mathbf{R}_k; \mathbf{L}_\star, \mathbf{R}_\star) &\leq \varepsilon_0 \tau^k \sigma_r(\mathbf{X}_\star), \\ \|\Delta_L \Sigma_\star^{1/2}\|_{2,\infty} \vee \|\Delta_R \Sigma_\star^{1/2}\|_{2,\infty} &\leq \sqrt{\frac{\mu r}{n}} \tau^k \sigma_r(\mathbf{X}_\star), \end{aligned}$$

then

$$\|\mathbf{X}_\star - \mathbf{X}_k\|_\infty \leq 3 \frac{\mu r}{n} \tau^k \sigma_r(\mathbf{X}_\star).$$

Proof. Firstly, by Assumption 1 and the assumption of this lemma, we have

$$\begin{aligned} \|\mathbf{R}_{\natural} \Sigma_\star^{-1/2}\|_{2,\infty} &\leq \|\Delta_R \Sigma_\star^{1/2}\|_{2,\infty} \|\Sigma_\star^{-1}\|_2 + \|\mathbf{L}_\star \Sigma_\star^{-1/2}\|_{2,\infty} \\ &\leq (\tau^k + 1) \sqrt{\frac{\mu r}{n}} \leq 2 \sqrt{\frac{\mu r}{n}}, \end{aligned}$$

given $\tau = 1 - 0.6\eta < 1$. Moreover, one can see

$$\begin{aligned} \|\mathbf{X}_\star - \mathbf{X}_k\|_\infty &= \|\Delta_L \mathbf{R}_{\natural}^\top + \mathbf{L}_\star \Delta_R^\top\|_\infty \leq \|\Delta_L \mathbf{R}_{\natural}^\top\|_\infty + \|\mathbf{L}_\star \Delta_R^\top\|_\infty \\ &\leq \|\Delta_L \Sigma_\star^{1/2}\|_{2,\infty} \|\mathbf{R}_{\natural} \Sigma_\star^{-1/2}\|_{2,\infty} + \|\mathbf{L}_\star \Sigma_\star^{-1/2}\|_{2,\infty} \|\Delta_R \Sigma_\star^{1/2}\|_{2,\infty} \\ &\leq \left(2 \sqrt{\frac{\mu r}{n}} + \sqrt{\frac{\mu r}{n}}\right) \sqrt{\frac{\mu r}{n}} \tau^k \sigma_r(\mathbf{X}_\star) \\ &= 3 \frac{\mu r}{n} \tau^k \sigma_r(\mathbf{X}_\star). \end{aligned}$$

This finishes the proof. $\square$

A.3 Proof of local linear convergence

We will show the local convergence of the proposed algorithm by first proving the claims stand at $(k+1)$ -th iteration if they stand at k -th iteration.

Lemma 11. *If*

$$\begin{aligned} \text{dist}(\mathbf{L}_k, \mathbf{R}_k; \mathbf{L}_\star, \mathbf{R}_\star) &\leq \varepsilon_0 \tau^k \sigma_r(\mathbf{X}_\star), \\ \|\Delta_L \Sigma_\star^{1/2}\|_{2,\infty} \vee \|\Delta_R \Sigma_\star^{1/2}\|_{2,\infty} &\leq \sqrt{\frac{\mu r}{n}} \tau^k \sigma_r(\mathbf{X}_\star), \end{aligned}$$

then

$$\text{dist}(\mathbf{L}_{k+1}, \mathbf{R}_{k+1}; \mathbf{L}_\star, \mathbf{R}_\star) \leq \varepsilon_0 \tau^{k+1} \sigma_r(\mathbf{X}_\star).$$

Proof. Since $\mathbf{Q}_{k+1}$ is the optimal alignment matrix between $(\mathbf{L}_{k+1}, \mathbf{R}_{k+1})$ and $(\mathbf{L}_\star, \mathbf{R}_\star)$, so

$$\begin{aligned} \text{dist}^2(\mathbf{L}_{k+1}, \mathbf{R}_{k+1}; \mathbf{L}_\star, \mathbf{R}_\star) &:= \|(\mathbf{L}_{k+1} \mathbf{Q}_{k+1} - \mathbf{L}_\star) \Sigma_\star^{1/2}\|_{\text{F}}^2 + \|(\mathbf{R}_{k+1} \mathbf{Q}_{k+1}^\top - \mathbf{R}_\star) \Sigma_\star^{1/2}\|_{\text{F}}^2 \\ &\leq \|(\mathbf{L}_{k+1} \mathbf{Q}_k - \mathbf{L}_\star) \Sigma_\star^{1/2}\|_{\text{F}}^2 + \|(\mathbf{R}_{k+1} \mathbf{Q}_k^\top - \mathbf{R}_\star) \Sigma_\star^{1/2}\|_{\text{F}}^2 \end{aligned}$$

We will focus on bounding the first term in this proof, and the second term can be bounded similarly.

Note that $\mathbf{L}_{k+1} \mathbf{R}_{k+1}^\top - \mathbf{X}_\star = \Delta_L \mathbf{R}_{k+1}^\top + \mathbf{L}_\star \Delta_R^\top$. We have

$$\begin{aligned} \mathbf{L}_{k+1} \mathbf{Q}_k - \mathbf{L}_\star &= \mathbf{L}_{k+1} - \eta(\mathbf{L}_{k+1} \mathbf{R}_{k+1}^\top - \mathbf{X}_\star + \mathbf{S}_{k+1} - \mathbf{S}_\star) \mathbf{R}_{k+1} (\mathbf{R}_{k+1}^\top \mathbf{R}_{k+1})^{-1} - \mathbf{L}_\star \\ &= \Delta_L - \eta(\mathbf{L}_{k+1} \mathbf{R}_{k+1}^\top - \mathbf{X}_\star) \mathbf{R}_{k+1} (\mathbf{R}_{k+1}^\top \mathbf{R}_{k+1})^{-1} - \eta \Delta_S \mathbf{R}_{k+1} (\mathbf{R}_{k+1}^\top \mathbf{R}_{k+1})^{-1} \\ &= (1 - \eta) \Delta_L - \eta \mathbf{L}_\star \Delta_R^\top \mathbf{R}_{k+1} (\mathbf{R}_{k+1}^\top \mathbf{R}_{k+1})^{-1} - \eta \Delta_S \mathbf{R}_{k+1} (\mathbf{R}_{k+1}^\top \mathbf{R}_{k+1})^{-1}. \end{aligned} \quad (15)$$

Thus,

$$\begin{aligned} &\|(\mathbf{L}_{k+1} \mathbf{Q}_k - \mathbf{L}_\star) \Sigma_\star^{1/2}\|_{\text{F}}^2 \\ &= \|(1 - \eta) \Delta_L \Sigma_\star^{1/2} - \eta \mathbf{L}_\star \Delta_R^\top \mathbf{R}_{k+1} (\mathbf{R}_{k+1}^\top \mathbf{R}_{k+1})^{-1} \Sigma_\star^{1/2}\|_{\text{F}}^2 - 2\eta(1 - \eta) \text{tr}(\Delta_S \mathbf{R}_{k+1} (\mathbf{R}_{k+1}^\top \mathbf{R}_{k+1})^{-1} \Sigma_\star \Delta_L^\top) \\ &\quad + 2\eta^2 \text{tr}(\Delta_S \mathbf{R}_{k+1} (\mathbf{R}_{k+1}^\top \mathbf{R}_{k+1})^{-1} \Sigma_\star (\mathbf{R}_{k+1}^\top \mathbf{R}_{k+1})^{-1} \mathbf{R}_{k+1}^\top \Delta_R \mathbf{L}_\star^\top) + \eta^2 \|\Delta_S \mathbf{R}_{k+1} (\mathbf{R}_{k+1}^\top \mathbf{R}_{k+1})^{-1} \Sigma_\star^{1/2}\|_{\text{F}}^2 \\ &:= \mathfrak{R}_1 - \mathfrak{R}_2 + \mathfrak{R}_3 + \mathfrak{R}_4 \end{aligned}$$

Bound of $\mathfrak{R}_1$. The component $\mathfrak{R}_1$ here is identical to $\mathfrak{R}_1$ in [12, Section D.1.1], and the bound of this term was shown therein. We will clear this bound further by applying Lemma 7, that is,

$$\begin{aligned} \mathfrak{R}_1 &\leq \left((1 - \eta)^2 + \frac{2\varepsilon_0}{1 - \varepsilon_0} \eta(1 - \eta) \right) \|\Delta_L \Sigma_\star^{1/2}\|_{\text{F}}^2 + \frac{2\varepsilon_0 + \varepsilon_0^2}{(1 - \varepsilon_0)^2} \eta^2 \|\Delta_R \Sigma_\star^{1/2}\|_{\text{F}}^2 \\ &\leq (1 - \eta)^2 \|\Delta_L \Sigma_\star^{1/2}\|_{\text{F}}^2 + \left((1 - \eta) \frac{2\varepsilon_0^3}{1 - \varepsilon_0} + \eta \frac{2\varepsilon_0^3 + \varepsilon_0^4}{(1 - \varepsilon_0)^2} \right) \eta \tau^{2k} \sigma_r^2(\mathbf{X}_\star). \end{aligned}$$

Bound of $\mathfrak{R}_2$. Lemma 5 implies $\Delta_S = \mathbf{S}_{k+1} - \mathbf{S}_\star$ is an α -sparse matrix. Thus, by Lemmata 6, 7, 8, 5, and 10, we have

$$\begin{aligned} |\text{tr}(\Delta_S \mathbf{R}_{k+1} (\mathbf{R}_{k+1}^\top \mathbf{R}_{k+1})^{-1} \Sigma_\star \Delta_L^\top)| &\leq \|\Delta_S\|_2 \|\mathbf{R}_{k+1} (\mathbf{R}_{k+1}^\top \mathbf{R}_{k+1})^{-1} \Sigma_\star \Delta_L^\top\|_* \\ &\leq \alpha n \sqrt{r} \|\Delta_S\|_\infty \|\mathbf{R}_{k+1} (\mathbf{R}_{k+1}^\top \mathbf{R}_{k+1})^{-1} \Sigma_\star \Delta_L^\top\|_{\text{F}} \\ &\leq 2\alpha n \sqrt{r} \|\mathbf{X}_k - \mathbf{X}_\star\|_\infty \|\mathbf{R}_{k+1} (\mathbf{R}_{k+1}^\top \mathbf{R}_{k+1})^{-1} \Sigma_\star^{1/2}\|_2 \|\Delta_L \Sigma_\star^{1/2}\|_{\text{F}} \\ &\leq 6\alpha \mu r^{1.5} \tau^{2k} \frac{\varepsilon_0}{1 - \varepsilon_0} \sigma_r^2(\mathbf{X}_\star). \end{aligned}$$

Hence,

$$|\mathfrak{R}_2| \leq 12\eta(1 - \eta) \alpha \mu r^{1.5} \tau^{2k} \frac{\varepsilon_0}{1 - \varepsilon_0} \sigma_r^2(\mathbf{X}_\star).$$

Bound of $\mathfrak{R}_3$. Similar to $\mathfrak{R}_2$, we have

$$\begin{aligned}
& |\operatorname{tr}(\Delta_S \mathbf{R}_{\mathfrak{q}} (\mathbf{R}_{\mathfrak{q}}^\top \mathbf{R}_{\mathfrak{q}})^{-1} \Sigma_\star (\mathbf{R}_{\mathfrak{q}}^\top \mathbf{R}_{\mathfrak{q}})^{-1} \mathbf{R}_{\mathfrak{q}}^\top \Delta_R \mathbf{L}_\star^\top)| \\
& \leq \|\Delta_S\|_2 \|\mathbf{R}_{\mathfrak{q}} (\mathbf{R}_{\mathfrak{q}}^\top \mathbf{R}_{\mathfrak{q}})^{-1} \Sigma_\star (\mathbf{R}_{\mathfrak{q}}^\top \mathbf{R}_{\mathfrak{q}})^{-1} \mathbf{R}_{\mathfrak{q}}^\top \Delta_R \mathbf{L}_\star^\top\|_* \\
& \leq \alpha n \sqrt{r} \|\Delta_S\|_\infty \|\mathbf{R}_{\mathfrak{q}} (\mathbf{R}_{\mathfrak{q}}^\top \mathbf{R}_{\mathfrak{q}})^{-1} \Sigma_\star (\mathbf{R}_{\mathfrak{q}}^\top \mathbf{R}_{\mathfrak{q}})^{-1} \mathbf{R}_{\mathfrak{q}}^\top \Delta_R \mathbf{L}_\star^\top\|_F \\
& \leq \alpha n \sqrt{r} \|\Delta_S\|_\infty \|\mathbf{R}_{\mathfrak{q}} (\mathbf{R}_{\mathfrak{q}}^\top \mathbf{R}_{\mathfrak{q}})^{-1} \Sigma_\star^{1/2}\|_2^2 \|\Delta_R \mathbf{L}_\star^\top\|_F \\
& \leq 2\alpha n \sqrt{r} \|\mathbf{X}_k - \mathbf{X}_\star\|_\infty \|\mathbf{R}_{\mathfrak{q}} (\mathbf{R}_{\mathfrak{q}}^\top \mathbf{R}_{\mathfrak{q}})^{-1} \Sigma_\star^{1/2}\|_2^2 \|\Delta_R \Sigma_\star^{1/2}\|_F \|\mathbf{U}_\star\|_2 \\
& \leq 6\alpha \mu r^{1.5} \tau^{2k} \frac{\varepsilon_0}{(1 - \varepsilon_0)^2} \sigma_r^2(\mathbf{X}_\star).
\end{aligned}$$

Hence,

$$|\mathfrak{R}_3| \leq 12\eta^2 \alpha \mu r^{1.5} \tau^{2k} \frac{\varepsilon_0}{(1 - \varepsilon_0)^2} \sigma_r^2(\mathbf{X}_\star).$$

Bound of $\mathfrak{R}_4$.

$$\begin{aligned}
\|\Delta_S \mathbf{R}_{\mathfrak{q}} (\mathbf{R}_{\mathfrak{q}}^\top \mathbf{R}_{\mathfrak{q}})^{-1} \Sigma_\star^{1/2}\|_F^2 & \leq r \|\Delta_S \mathbf{R}_{\mathfrak{q}} (\mathbf{R}_{\mathfrak{q}}^\top \mathbf{R}_{\mathfrak{q}})^{-1} \Sigma_\star^{1/2}\|_2^2 \\
& \leq r \|\Delta_S\|_2^2 \|\mathbf{R}_{\mathfrak{q}} (\mathbf{R}_{\mathfrak{q}}^\top \mathbf{R}_{\mathfrak{q}})^{-1} \Sigma_\star^{1/2}\|_2^2 \\
& \leq 4\alpha^2 n^2 r \|\mathbf{X}_k - \mathbf{X}_\star\|_\infty^2 \|\mathbf{R}_{\mathfrak{q}} (\mathbf{R}_{\mathfrak{q}}^\top \mathbf{R}_{\mathfrak{q}})^{-1} \Sigma_\star^{1/2}\|_2^2 \\
& \leq 36\alpha^2 \mu^2 r^3 \tau^{2k} \frac{1}{(1 - \varepsilon_0)^2} \sigma_r^2(\mathbf{X}_\star).
\end{aligned}$$

Hence,

$$\mathfrak{R}_4 \leq 36\eta^2 \alpha^2 \mu^2 r^3 \tau^{2k} \frac{1}{(1 - \varepsilon_0)^2} \sigma_r^2(\mathbf{X}_\star).$$

Combine all the bounds together, we have

$$\begin{aligned}
& \|(\mathbf{L}_{k+1} \mathbf{Q}_k - \mathbf{L}_\star) \Sigma_\star^{1/2}\|_F^2 \\
& \leq (1 - \eta)^2 \|\Delta_L \Sigma_\star^{1/2}\|_F^2 + \left((1 - \eta) \frac{2\varepsilon_0^3}{1 - \varepsilon_0} + \eta \frac{2\varepsilon_0^3 + \varepsilon_0^4}{(1 - \varepsilon_0)^2} \right) \eta \tau^{2k} \sigma_r^2(\mathbf{X}_\star) \\
& \quad + 12\eta(1 - \eta) \alpha \mu r^{1.5} \tau^{2k} \frac{\varepsilon_0}{1 - \varepsilon_0} \sigma_r^2(\mathbf{X}_\star) \\
& \quad + 12\eta^2 \alpha \mu r^{1.5} \tau^{2k} \frac{\varepsilon_0}{(1 - \varepsilon_0)^2} \sigma_r^2(\mathbf{X}_\star) \\
& \quad + 36\alpha^2 \mu^2 r^3 \tau^{2k} \frac{1}{(1 - \varepsilon_0)^2} \sigma_r^2(\mathbf{X}_\star),
\end{aligned}$$

and a similar bound can be computed for $\|(\mathbf{R}_{k+1} \mathbf{Q}_k^{-\top} - \mathbf{R}_\star) \Sigma_\star^{1/2}\|_F^2$. Add together, we have

$$\begin{aligned}
& \operatorname{dist}^2(\mathbf{L}_{k+1}, \mathbf{R}_{k+1}; \mathbf{L}_\star, \mathbf{R}_\star) \\
& \leq \|(\mathbf{L}_{k+1} \mathbf{Q}_k - \mathbf{L}_\star) \Sigma_\star^{1/2}\|_F^2 + \|(\mathbf{R}_{k+1} \mathbf{Q}_k^{-\top} - \mathbf{R}_\star) \Sigma_\star^{1/2}\|_F^2 \\
& \leq (1 - \eta)^2 \left(\|\Delta_L \Sigma_\star^{1/2}\|_F^2 + \|\Delta_R \Sigma_\star^{1/2}\|_F^2 \right) + 2 \left((1 - \eta) \frac{2\varepsilon_0^3}{1 - \varepsilon_0} + \eta \frac{2\varepsilon_0^3 + \varepsilon_0^4}{(1 - \varepsilon_0)^2} \right) \eta \tau^{2k} \sigma_r^2(\mathbf{X}_\star) \\
& \quad + 24\eta(1 - \eta) \alpha \mu r^{1.5} \tau^{2k} \frac{\varepsilon_0}{1 - \varepsilon_0} \sigma_r^2(\mathbf{X}_\star) \\
& \quad + 24\eta^2 \alpha \mu r^{1.5} \tau^{2k} \frac{\varepsilon_0}{(1 - \varepsilon_0)^2} \sigma_r^2(\mathbf{X}_\star) \\
& \quad + 72\alpha^2 \mu^2 r^3 \tau^{2k} \frac{1}{(1 - \varepsilon_0)^2} \sigma_r^2(\mathbf{X}_\star) \\
& \leq \left((1 - \eta)^2 + 2 \left((1 - \eta) \frac{2\varepsilon_0}{1 - \varepsilon_0} + \eta \frac{2\varepsilon_0 + \varepsilon_0^2}{(1 - \varepsilon_0)^2} \right) \eta + 24\eta(1 - \eta) \alpha \mu r^{1.5} \frac{1}{\varepsilon_0(1 - \varepsilon_0)} \right) \eta \tau^{2k} \sigma_r^2(\mathbf{X}_\star)
\end{aligned}$$

$$\begin{aligned}
& + 24\eta^2 \alpha \mu r^{1.5} \frac{1}{\varepsilon_0(1-\varepsilon_0)^2} + 72\alpha^2 \mu^2 r^3 \frac{1}{\varepsilon_0^2(1-\varepsilon_0)^2} \Big) \varepsilon_0^2 \tau^{2k} \sigma_r^2(\mathbf{X}_\star) \\
& \leq (1 - 0.6\eta)^2 \varepsilon_0^2 \tau^{2k} \sigma_r^2(\mathbf{X}_\star),
\end{aligned} \tag{16}$$

where we use the fact $\|\Delta_L \Sigma_\star^{1/2}\|_{\text{F}}^2 + \|\Delta_R \Sigma_\star^{1/2}\|_{\text{F}}^2 =: \text{dist}^2(\mathbf{L}_k, \mathbf{R}_k; \mathbf{L}_\star, \mathbf{R}_\star) \leq \varepsilon_0^2 \tau^{2k} \sigma_r^2(\mathbf{X}_\star)$ in the second step, and the last step use $\varepsilon_0 = 0.02$, $\alpha \leq \frac{1}{10^4 \mu r^{1.5}}$, and $\frac{1}{4} \leq \eta \leq \frac{8}{9}$. The proof is finished by substituting $\tau = 1 - 0.6\eta$. $\square$

Lemma 12. *If*

$$\begin{aligned}
& \text{dist}(\mathbf{L}_k, \mathbf{R}_k; \mathbf{L}_\star, \mathbf{R}_\star) \leq \varepsilon_0 \tau^k \sigma_r(\mathbf{X}_\star), \\
& \|\Delta_L \Sigma_\star^{1/2}\|_{2,\infty} \vee \|\Delta_R \Sigma_\star^{1/2}\|_{2,\infty} \leq \sqrt{\frac{\mu r}{n}} \tau^k \sigma_r(\mathbf{X}_\star),
\end{aligned}$$

then

$$\|(\mathbf{L}_{k+1} \mathbf{Q}_{k+1} - \mathbf{L}_\star) \Sigma_\star^{1/2}\|_{2,\infty} \vee \|(\mathbf{R}_{k+1} \mathbf{Q}_{k+1}^\top - \mathbf{R}_\star) \Sigma_\star^{1/2}\|_{2,\infty} \leq \sqrt{\frac{\mu r}{n}} \tau^{k+1} \sigma_r(\mathbf{X}_\star).$$

Proof. Using (15) again, we have

$$\begin{aligned}
& \|(\mathbf{L}_{k+1} \mathbf{Q}_k - \mathbf{L}_\star) \Sigma_\star^{1/2}\|_{2,\infty} \\
& \leq (1 - \eta) \|\Delta_L \Sigma_\star^{1/2}\|_{2,\infty} + \eta \|\mathbf{L}_\star \Delta_R^\top \mathbf{R}_k (\mathbf{R}_k^\top \mathbf{R}_k)^{-1} \Sigma_\star^{1/2}\|_{2,\infty} + \eta \|\Delta_S \mathbf{R}_k (\mathbf{R}_k^\top \mathbf{R}_k)^{-1} \Sigma_\star^{1/2}\|_{2,\infty} \\
& := \mathfrak{T}_1 + \mathfrak{T}_2 + \mathfrak{T}_3.
\end{aligned}$$

Bound of $\mathfrak{T}_1$. $\mathfrak{T}_1 \leq (1 - \eta) \sqrt{\frac{\mu r}{n}} \tau^k \sigma_r(\mathbf{X}_\star)$ is directly followed by the assumption of this lemma.

Bound of $\mathfrak{T}_2$. Assumption 1 implies $\|\mathbf{L}_\star \Sigma_\star^{-1/2}\|_{2,\infty} \leq \sqrt{\frac{\mu r}{n}}$, Lemma 7 implies $\|\Delta_R \Sigma_\star^{1/2}\|_2 \leq \tau^k \varepsilon_0$, and Lemma 8 implies Together, we have

$$\begin{aligned}
\mathfrak{T}_2 & \leq \eta \|\mathbf{L}_\star \Sigma_\star^{-1/2}\|_{2,\infty} \|\Delta_R \Sigma_\star^{1/2}\|_2 \|\mathbf{R}_k (\mathbf{R}_k^\top \mathbf{R}_k)^{-1} \Sigma_\star^{1/2}\|_2 \\
& \leq \eta \frac{\varepsilon_0}{1 - \varepsilon_0} \sqrt{\frac{\mu r}{n}} \tau^k \sigma_r(\mathbf{X}_\star).
\end{aligned}$$

Bound of $\mathfrak{T}_3$. By Lemma 5, $\text{supp}(\Delta_S) \subseteq \text{supp}(\mathbf{S}_\star)$, which implies that Δ_S is an α -sparse matrix. Thus, by Lemma 6, we get

$$\begin{aligned}
\mathfrak{T}_3 & \leq \eta \|\Delta_S\|_{2,\infty} \|\mathbf{R}_k (\mathbf{R}_k^\top \mathbf{R}_k)^{-1} \Sigma_\star^{1/2}\|_2 \\
& \leq \eta \frac{\sqrt{\alpha n}}{1 - \varepsilon_0} \|\Delta_S\|_\infty \\
& \leq 2\eta \frac{\sqrt{\alpha n}}{1 - \varepsilon_0} \|\mathbf{X}_\star - \mathbf{X}_k\|_\infty \\
& \leq 6\eta \frac{\sqrt{\alpha \mu r}}{1 - \varepsilon_0} \sqrt{\frac{\mu r}{n}} \tau^k \sigma_r(\mathbf{X}_\star).
\end{aligned}$$

where the last two steps use Lemmata 5 and 10 respectively. Put together, we obtain

$$\begin{aligned}
\|(\mathbf{L}_{k+1} \mathbf{Q}_k - \mathbf{L}_\star) \Sigma_\star^{1/2}\|_{2,\infty} & \leq \mathfrak{T}_1 + \mathfrak{T}_2 + \mathfrak{T}_3 \\
& \leq \left(1 - \eta + \eta \frac{\varepsilon_0}{1 - \varepsilon_0} + 6\eta \frac{\sqrt{\alpha \mu r}}{1 - \varepsilon_0}\right) \sqrt{\frac{\mu r}{n}} \tau^k \sigma_r(\mathbf{X}_\star) \\
& \leq \left(1 - \eta \left(1 - \frac{\varepsilon_0}{1 - \varepsilon_0} - 6 \frac{\sqrt{\alpha \mu r}}{1 - \varepsilon_0}\right)\right) \sqrt{\frac{\mu r}{n}} \tau^k \sigma_r(\mathbf{X}_\star). \tag{17}
\end{aligned}$$

In addition, we also have

$$\|(\mathbf{L}_{k+1} \mathbf{Q}_k - \mathbf{L}_\star) \Sigma_\star^{-1/2}\|_{2,\infty} \leq \left(1 - \eta \left(1 - \frac{\varepsilon_0}{1 - \varepsilon_0} - 6 \frac{\sqrt{\alpha \mu r}}{1 - \varepsilon_0}\right)\right) \sqrt{\frac{\mu r}{n}} \tau^k. \tag{18}$$

Bound with $\mathbf{Q}_{k+1}$. Note that $\mathbf{Q}$'s are the best align matrices under Frobenius norm but this is not necessary true under $\ell_{2,\infty}$ norm. So we must show the bound of $\|(\mathbf{L}_{k+1}\mathbf{Q}_{k+1} - \mathbf{L}_*)\Sigma_*^{1/2}\|_{2,\infty}$ directly. Note that $\mathbf{Q}_{k+1}$ does exist, according to Lemmata 11 and 2. Applying (17), (18) and Lemma 9, we have

$$\begin{aligned}
& \|(\mathbf{L}_{k+1}\mathbf{Q}_{k+1} - \mathbf{L}_*)\Sigma_*^{1/2}\|_{2,\infty} \\
& \leq \|(\mathbf{L}_{k+1}\mathbf{Q}_k - \mathbf{L}_*)\Sigma_*^{1/2}\|_{2,\infty} + \|\mathbf{L}_{k+1}(\mathbf{Q}_{k+1} - \mathbf{Q}_k)\Sigma_*^{1/2}\|_{2,\infty} \\
& = \|(\mathbf{L}_{k+1}\mathbf{Q}_k - \mathbf{L}_*)\Sigma_*^{1/2}\|_{2,\infty} + \|\mathbf{L}_{k+1}\mathbf{Q}_k\Sigma_*^{-1/2}\Sigma_*^{1/2}\mathbf{Q}_k^{-1}(\mathbf{Q}_{k+1} - \mathbf{Q}_k)\Sigma_*^{1/2}\|_{2,\infty} \\
& \leq \|(\mathbf{L}_{k+1}\mathbf{Q}_k - \mathbf{L}_*)\Sigma_*^{1/2}\|_{2,\infty} + \|\mathbf{L}_{k+1}\mathbf{Q}_k\Sigma_*^{-1/2}\|_{2,\infty}\|\Sigma_*^{1/2}\mathbf{Q}_k^{-1}(\mathbf{Q}_{k+1} - \mathbf{Q}_k)\Sigma_*^{1/2}\|_2 \\
& \leq \|(\mathbf{L}_{k+1}\mathbf{Q}_k - \mathbf{L}_*)\Sigma_*^{1/2}\|_{2,\infty} \\
& \quad + \left(\|(\mathbf{L}_{k+1}\mathbf{Q}_k - \mathbf{L}_*)\Sigma_*^{-1/2}\|_{2,\infty} + \|\mathbf{L}_*\Sigma_*^{-1/2}\|_{2,\infty} \right) \|\Sigma_*^{1/2}\mathbf{Q}_k^{-1}(\mathbf{Q}_{k+1} - \mathbf{Q}_k)\Sigma_*^{1/2}\|_2 \\
& \leq \left(1 - \eta \left(1 - \frac{\varepsilon_0}{1 - \varepsilon_0} - 6\frac{\sqrt{\alpha\mu r}}{1 - \varepsilon_0} \right) + \frac{2\varepsilon_0}{1 - \varepsilon_0} \left(2 - \eta \left(1 - \frac{\varepsilon_0}{1 - \varepsilon_0} - 6\frac{\sqrt{\alpha\mu r}}{1 - \varepsilon_0} \right) \right) \right) \\
& \quad \sqrt{\frac{\mu r}{n}}\tau^k\sigma_r(\mathbf{X}_*) \\
& \leq (1 - 0.6\eta)\sqrt{\frac{\mu r}{n}}\tau^k\sigma_r(\mathbf{X}_*),
\end{aligned}$$

where the last step use $\varepsilon_0 = 0.02$, $\alpha \leq \frac{1}{10^4\mu r^{1.5}}$, and $\frac{1}{4} \leq \eta \leq \frac{8}{9}$. Similar result can be computed for $\|(\mathbf{R}_{k+1}\mathbf{Q}_{k+1}^\top - \mathbf{R}_*)\Sigma_*^{1/2}\|_{2,\infty}$. The proof is finished by substituting $\tau = 1 - 0.6\eta$. $\square$

Now we have all the ingredients for proving the theorem of local linear convergence, i.e., Theorem 3.

Proof of Theorem 3. This proof is done by induction.

Base case. Since $\tau^0 = 1$, the assumed initial conditions satisfy the base case at $k = 0$.

Induction step. At the k -th iteration, we assume the conditions

$$\begin{aligned}
& \text{dist}(\mathbf{L}_k, \mathbf{R}_k; \mathbf{L}_*, \mathbf{R}_*) \leq \varepsilon_0\tau^k\sigma_r(\mathbf{X}_*), \\
& \|(\mathbf{L}_k\mathbf{Q}_k - \mathbf{L}_*)\Sigma_*^{1/2}\|_{2,\infty} \vee \|(\mathbf{R}_k\mathbf{Q}_k^\top - \mathbf{R}_*)\Sigma_*^{1/2}\|_{2,\infty} \leq \sqrt{\frac{\mu r}{n}}\tau^k\sigma_r(\mathbf{X}_*)
\end{aligned}$$

hold, then by Lemmata 11 and 12,

$$\begin{aligned}
& \text{dist}(\mathbf{L}_{k+1}, \mathbf{R}_{k+1}; \mathbf{L}_*, \mathbf{R}_*) \leq \varepsilon_0\tau^{k+1}\sigma_r(\mathbf{X}_*), \\
& \|(\mathbf{L}_{k+1}\mathbf{Q}_{k+1} - \mathbf{L}_*)\Sigma_*^{1/2}\|_{2,\infty} \vee \|(\mathbf{R}_{k+1}\mathbf{Q}_{k+1}^\top - \mathbf{R}_*)\Sigma_*^{1/2}\|_{2,\infty} \leq \sqrt{\frac{\mu r}{n}}\tau^{k+1}\sigma_r(\mathbf{X}_*)
\end{aligned}$$

also hold. This finishes the proof. $\square$

A.4 Proof of guaranteed initialization

Now we show the outputs of the initialization step in Algorithm 1 satisfy the initial conditions required by Theorem 3.

Proof of Theorem 4. Firstly, by Assumption 1, we obtain

$$\|\mathbf{X}_*\|_\infty \leq \|\mathbf{U}_*\|_{2,\infty}\|\Sigma_*\|_2\|\mathbf{V}_*\|_{2,\infty} \leq \frac{\mu r}{n}\sigma_1(\mathbf{X}_*).$$

Invoking Lemma 5 with $\mathbf{X}_{-1} = \mathbf{0}$, we have

$$\|\mathbf{S}_* - \mathbf{S}_0\|_\infty \leq 2\frac{\mu r}{n}\sigma_1(\mathbf{X}_*) \quad \text{and} \quad \text{supp}(\mathbf{S}_0) \subseteq \text{supp}(\mathbf{S}_*), \quad (19)$$

which implies $\mathbf{S}_\star - \mathbf{S}_0$ is an α -sparse matrix. Applying Lemma 6, we have

$$\|\mathbf{S}_\star - \mathbf{S}_0\|_2 \leq \alpha n \|\mathbf{S}_\star - \mathbf{S}_0\|_\infty \leq 2\alpha\mu r \sigma_1(\mathbf{X}_\star) = 2\alpha\mu r \kappa \sigma_r(\mathbf{X}_\star).$$

Since $\mathbf{X}_0 = \mathbf{L}_0 \mathbf{R}_0^\top$ is the best rank- r approximation of $\mathbf{Y} - \mathbf{S}_0$, so

$$\begin{aligned} \|\mathbf{X}_\star - \mathbf{X}_0\|_2 &\leq \|\mathbf{X}_\star - (\mathbf{Y} - \mathbf{S}_0)\|_2 + \|(\mathbf{Y} - \mathbf{S}_0) - \mathbf{X}_0\|_2 \\ &\leq 2\|\mathbf{X}_\star - (\mathbf{Y} - \mathbf{S}_0)\|_2 \\ &= 2\|\mathbf{S}_\star - \mathbf{S}_0\|_2 \\ &\leq 4\alpha\mu r \kappa \sigma_r(\mathbf{X}_\star), \end{aligned}$$

where the equality uses the definition $\mathbf{Y} = \mathbf{X}_\star + \mathbf{S}_\star$. By [12, Lemma 11], we obtain

$$\begin{aligned} \text{dist}(\mathbf{L}_0, \mathbf{R}_0; \mathbf{L}_\star, \mathbf{R}_\star) &\leq \sqrt{\sqrt{2} + 1} \|\mathbf{X}_\star - \mathbf{X}_0\|_F \\ &\leq \sqrt{(\sqrt{2} + 1)2r} \|\mathbf{X}_\star - \mathbf{X}_0\|_2 \\ &\leq 10\alpha\mu r^{1.5} \kappa \sigma_r(\mathbf{X}_\star), \end{aligned}$$

where we use the fact that $\mathbf{X}_\star - \mathbf{X}_0$ has at most rank- $2r$. Given $\varepsilon_0 = 10c_0$ and $\alpha \leq \frac{c_0}{\mu r^{1.5} \kappa}$, our first claim

$$\text{dist}(\mathbf{L}_0, \mathbf{R}_0; \mathbf{L}_\star, \mathbf{R}_\star) \leq 10c_0 \sigma_r(\mathbf{X}_\star) \quad (20)$$

is proved.

Let $\varepsilon_0 := 10c_0$. Now, we will prove the second claim:

$$\|\Delta_L \Sigma_\star^{1/2}\|_{2,\infty} \vee \|\Delta_R \Sigma_\star^{1/2}\|_{2,\infty} \leq \sqrt{\frac{\mu r}{n}} \sigma_r(\mathbf{X}_\star)$$

where $\Delta_L := \mathbf{L}_0 \mathbf{Q}_0 - \mathbf{L}_\star$ and $\Delta_R := \mathbf{R}_0 \mathbf{Q}_0^{-\top} - \mathbf{R}_\star$. For ease of notation, we also denote $\mathbf{L}_{\natural} = \mathbf{L}_0 \mathbf{Q}_0$, $\mathbf{R}_{\natural} = \mathbf{R}_0 \mathbf{Q}_0^{-\top}$, and $\Delta_S = \mathbf{S}_0 - \mathbf{S}_\star$ in the rest of this proof.

We will work on $\|\Delta \Sigma_\star^{1/2}\|_{2,\infty}$ first, and $\|\Delta \Sigma_\star^{1/2}\|_{2,\infty}$ can be bounded similarly.

Since $\mathbf{U}_0 \Sigma_0 \mathbf{V}_0^\top = \mathcal{D}_r(\mathbf{Y} - \mathbf{S}_0) = \mathcal{D}_r(\mathbf{X}_\star - \Delta_S)$, so

$$\begin{aligned} \mathbf{L}_0 &= \mathbf{U}_0 \Sigma_0^{1/2} = (\mathbf{X}_\star - \Delta_S) \mathbf{V}_0 \Sigma_0^{-1/2} \\ &= (\mathbf{X}_\star - \Delta_S) \mathbf{R}_0 \Sigma_0^{-1} \\ &= (\mathbf{X}_\star - \Delta_S) \mathbf{R}_0 (\mathbf{R}_0^\top \mathbf{R}_0)^{-1}. \end{aligned}$$

Multiplying $\mathbf{Q}_0 \Sigma_\star^{1/2}$ on both sides, we have

$$\begin{aligned} \mathbf{L}_{\natural} \Sigma_\star^{1/2} &= \mathbf{L}_0 \mathbf{Q}_0 \Sigma_\star^{1/2} = (\mathbf{X}_\star - \Delta_S) \mathbf{R}_0 (\mathbf{R}_0^\top \mathbf{R}_0)^{-1} \mathbf{Q}_0 \Sigma_\star^{1/2} \\ &= (\mathbf{X}_\star - \Delta_S) \mathbf{R}_{\natural} (\mathbf{R}_{\natural}^\top \mathbf{R}_{\natural})^{-1} \Sigma_\star^{1/2}. \end{aligned}$$

Subtracting $\mathbf{X}_\star \mathbf{R}_{\natural} (\mathbf{R}_{\natural}^\top \mathbf{R}_{\natural})^{-1} \Sigma_\star^{1/2}$ on both sides, we have

$$\begin{aligned} \mathbf{L}_{\natural} \Sigma_\star^{1/2} - \mathbf{X}_\star \mathbf{R}_{\natural} (\mathbf{R}_{\natural}^\top \mathbf{R}_{\natural})^{-1} \Sigma_\star^{1/2} &= (\mathbf{X}_\star - \Delta_S) \mathbf{R}_{\natural} (\mathbf{R}_{\natural}^\top \mathbf{R}_{\natural})^{-1} \Sigma_\star^{1/2} - \mathbf{X}_\star \mathbf{R}_{\natural} (\mathbf{R}_{\natural}^\top \mathbf{R}_{\natural})^{-1} \Sigma_\star^{1/2} \\ \Delta_L \Sigma_\star^{1/2} + \mathbf{X}_\star \Delta_R^\top \mathbf{R}_{\natural} (\mathbf{R}_{\natural}^\top \mathbf{R}_{\natural})^{-1} \Sigma_\star^{1/2} &= -\Delta_S \mathbf{R}_{\natural} (\mathbf{R}_{\natural}^\top \mathbf{R}_{\natural})^{-1} \Sigma_\star^{1/2}, \end{aligned}$$

where the left operand of last step uses the fact $\mathbf{L}_\star \Sigma_\star^{1/2} = \mathbf{L}_\star \mathbf{R}_{\natural}^\top \mathbf{R}_{\natural} (\mathbf{R}_{\natural}^\top \mathbf{R}_{\natural})^{-1} \Sigma_\star^{1/2}$. Thus,

$$\begin{aligned} \|\Delta_L \Sigma_\star^{1/2}\|_{2,\infty} &\leq \|\mathbf{X}_\star \Delta_R^\top \mathbf{R}_{\natural} (\mathbf{R}_{\natural}^\top \mathbf{R}_{\natural})^{-1} \Sigma_\star^{1/2}\|_{2,\infty} + \|\Delta_S \mathbf{R}_{\natural} (\mathbf{R}_{\natural}^\top \mathbf{R}_{\natural})^{-1} \Sigma_\star^{1/2}\|_{2,\infty} \\ &:= \mathfrak{J}_1 + \mathfrak{J}_2 \end{aligned}$$

Bound of $\mathfrak{J}_1$. By Assumption 1, we get

$$\begin{aligned} \mathfrak{J}_1 &\leq \|\mathbf{X}_\star \Sigma_\star^{-1/2}\|_{2,\infty} \|\Delta_R \Sigma_\star^{1/2}\|_2 \|\mathbf{R}_{\natural} (\mathbf{R}_{\natural}^\top \mathbf{R}_{\natural})^{-1} \Sigma_\star^{1/2}\|_2 \\ &\leq \sqrt{\frac{\mu r}{n}} \frac{\varepsilon_0}{1 - \varepsilon_0} \sigma_r(\mathbf{X}_\star) \end{aligned}$$

where Lemma 7 implies $\|\Delta_R \Sigma_\star^{1/2}\|_2 \leq \varepsilon_0 \sigma_r(\mathbf{X}_\star)$, and Lemma 8 implies $\|\mathbf{R}_{\natural} (\mathbf{R}_{\natural}^\top \mathbf{R}_{\natural})^{-1} \Sigma_\star^{1/2}\|_2 \leq \frac{1}{1 - \varepsilon_0}$, given (20) holds.

Bound of $\mathfrak{J}_2$. (19) implies Δ_S is α -sparse. Moreover, by (20), Lemmata 6 and 8, we have

$$\begin{aligned}
\mathfrak{J}_2 &\leq \|\Delta_S\|_{1,\infty} \|\mathbf{R}_{\mathfrak{d}} \Sigma_\star^{-1/2}\|_{2,\infty} \|\Sigma_\star^{1/2} (\mathbf{R}_{\mathfrak{d}}^\top \mathbf{R}_{\mathfrak{d}})^{-1} \Sigma_\star^{1/2}\|_2 \\
&\leq \alpha n \|\Delta_S\|_\infty \|\mathbf{R}_{\mathfrak{d}} \Sigma_\star^{-1/2}\|_{2,\infty} \|\mathbf{R}_{\mathfrak{d}} (\mathbf{R}_{\mathfrak{d}}^\top \mathbf{R}_{\mathfrak{d}})^{-1} \Sigma_\star^{1/2}\|_2^2 \\
&\leq \alpha n \frac{2\mu r}{n} \sigma_1(\mathbf{X}_\star) \frac{1}{(1-\varepsilon_0)^2} \|\mathbf{R}_{\mathfrak{d}} \Sigma_\star^{-1/2}\|_{2,\infty} \\
&\leq \frac{2\alpha\mu r\kappa}{(1-\varepsilon_0)^2} \left(\sqrt{\frac{\mu r}{n}} + \|\Delta_R \Sigma_\star^{-1/2}\|_{2,\infty} \right) \sigma_r(\mathbf{X}_\star)
\end{aligned}$$

where the first step uses that $\|\mathbf{AB}\|_{2,\infty} \leq \|\mathbf{A}\|_{1,\infty} \|\mathbf{B}\|_{2,\infty}$ for any matrices. Note that $\|\Delta_R \Sigma_\star^{-1/2}\|_{2,\infty} \leq \frac{\|\Delta_R \Sigma_\star^{1/2}\|_{2,\infty}}{\sigma_r(\mathbf{X}_\star)}$. Hence,

$$\|\Delta_L \Sigma_\star^{1/2}\|_{2,\infty} \leq \left(\frac{\varepsilon_0}{1-\varepsilon_0} + \frac{2\alpha\mu r\kappa}{(1-\varepsilon_0)^2} \right) \sqrt{\frac{\mu r}{n}} \sigma_r(\mathbf{X}_\star) + \frac{2\alpha\mu r\kappa}{(1-\varepsilon_0)^2} \|\Delta_R \Sigma_\star^{1/2}\|_{2,\infty},$$

and similarly one can see

$$\|\Delta_R \Sigma_\star^{1/2}\|_{2,\infty} \leq \left(\frac{\varepsilon_0}{1-\varepsilon_0} + \frac{2\alpha\mu r\kappa}{(1-\varepsilon_0)^2} \right) \sqrt{\frac{\mu r}{n}} \sigma_r(\mathbf{X}_\star) + \frac{2\alpha\mu r\kappa}{(1-\varepsilon_0)^2} \|\Delta_L \Sigma_\star^{1/2}\|_{2,\infty}.$$

Therefore, substituting $\varepsilon_0 = 10c_0$ gives

$$\begin{aligned}
&\|\Delta_L \Sigma_\star^{1/2}\|_{2,\infty} \vee \|\Delta_R \Sigma_\star^{1/2}\|_{2,\infty} \\
&\leq \frac{(1-\varepsilon_0)^2}{(1-\varepsilon_0)^2 - 2\alpha\mu r\kappa} \left(\frac{\varepsilon_0}{1-\varepsilon_0} + \frac{2\alpha\mu r\kappa}{(1-\varepsilon_0)^2} \right) \sqrt{\frac{\mu r}{n}} \sigma_r(\mathbf{X}_\star) \\
&\leq \frac{(1-10c_0)^2}{(1-10c_0)^2 - 2c_0} \left(\frac{10c_0}{1-10c_0} + \frac{2c_0}{(1-10c_0)^2} \right) \sqrt{\frac{\mu r}{n}} \sigma_r(\mathbf{X}_\star) \\
&\leq \sqrt{\frac{\mu r}{n}} \sigma_r(\mathbf{X}_\star),
\end{aligned}$$

as long as $c_0 \leq \frac{1}{35}$. This finishes the proof. $\square$

B Complexity of LRPCA

We provide the breakdown of LRPCA's computational complexity:

1. Compute $\mathbf{L}_k \mathbf{R}_k^\top$: n -by- r matrix times r -by- n matrix— $n^2 r$ flops.
2. Compute $\mathbf{Y} - \mathbf{L}_k \mathbf{R}_k^\top$: n -by- n matrix minus n -by- n matrix— n^2 flops.
3. Soft-thresholding on $\mathbf{Y} - \mathbf{L}_k \mathbf{R}_k^\top$: one pass on n -by- n matrix— n^2 flops.
4. Compute $\mathbf{L}_k \mathbf{R}_k^\top + \mathbf{S}_{k+1} - \mathbf{Y} = \mathbf{S}_{k+1} - (\mathbf{Y} - \mathbf{L}_k \mathbf{R}_k^\top)$: n -by- n matrix minus n -by- n matrix— n^2 flops.
5. Compute $\mathbf{R}_k^\top \mathbf{R}_k$: r -by- n matrix times n -by- r matrix— nr^2 flops.
6. Compute $(\mathbf{R}_k^\top \mathbf{R}_k)^{-1}$: invert a r -by- r matrix— $\mathcal{O}(r^3)$ flops.
7. Compute $\mathbf{R}_k (\mathbf{R}_k^\top \mathbf{R}_k)^{-1}$: n -by- r matrix times r -by- r matrix— nr^2 flops.
8. Compute $(\mathbf{L}_k \mathbf{R}_k^\top + \mathbf{S}_{k+1} - \mathbf{Y}) \cdot \mathbf{R}_k (\mathbf{R}_k^\top \mathbf{R}_k)^{-1}$: n -by- n matrix times n -by- r matrix— $n^2 r$ flops.
9. Compute $\mathbf{L}_{k+1} = \mathbf{L}_k - \zeta_{k+1} (\mathbf{L}_k \mathbf{R}_k^\top + \mathbf{S}_{k+1} - \mathbf{Y}) \mathbf{R}_k (\mathbf{R}_k^\top \mathbf{R}_k)^{-1}$: n -by- r matrix minus scalar times n -by- r matrix— $2nr$ flops.
10. Repeat step 5 - 9 for computing $\mathbf{R}_{k+1}$ —another $2nr^2 + \mathcal{O}(r^3) + n^2 r + 2nr$ flops.

In total, LRPCA costs $3n^2 r + 3n^2 + \mathcal{O}(nr^2)$ flops per iteration provided $r \ll n$. Note that we count abc flops for computing an a -by- b matrix times a b -by- c matrix in the above complexity calculation. Some may argue that this matrix product should take $2abc$ flops. The per-iteration complexity can be rectified to $6n^2 r + 3n^2 + \mathcal{O}(nr^2)$ flops if the reader prefers the latter opinion.

C Additional numerical results

C.1 Setup details

Random instance generation. We follow the setup in [8, 10] to generate synthetic data. Each observation signal $\mathbf{Y}_* \in \mathbb{R}^{n \times n}$ is generated by $\mathbf{Y}_* = \mathbf{X}_* + \mathbf{S}_*$. The underlying low-rank matrix $\mathbf{X}_*$ is generated with $\mathbf{X}_* = \mathbf{L}_* \mathbf{R}_*^\top$ where $\mathbf{L}_*, \mathbf{R}_* \in \mathbb{R}^{n \times r}$ have elements drawn i.i.d from zero-mean Gaussian distribution with variance $1/n$. Non-zero locations of the underlying sparse matrix $\mathbf{S}_*$ is uniformly and independently sampled without replacement. The magnitudes of the non-zeros of $\mathbf{S}_*$ are sampled i.i.d from the uniform distribution over the interval $[-\mathbb{E}|\mathbf{X}_*|_{i,j}|, \mathbb{E}|\mathbf{X}_*|_{i,j}|]$.

Video instances preprocessing. To accelerate the training process, we first change the RGB videos in the VIRAT dataset to gray videos and then downsample the videos by a fraction of 4. All training videos are cut to sub-videos with number of frames no more than 1000, testing videos are not cut.

Details in training. In the layer-wise training phase, we adopt SGD with batch size 1; in the parameter (β, ϕ) searching phase (i.e., RNN training), we adopt grid search with grid size 0.1. In synthetic data experiments, the ground truth $\mathbf{X}_*$ is known after each instance is generated. Thus, the training pair $(\mathbf{Y}, \mathbf{X}_*)$ is easy to obtain. We generate a new instance in each step of SGD in the layer-wise training phase and generate 20 instances for the grid search phase. The testing set is separately generated and consists of 50 instances. In the video experiment, the underlying ground truth $\mathbf{X}_*$ is unknown. We solve each training video with a classic RPCA algorithm [6] (without learning) to precision 10^{-5} and use that solution as $\mathbf{X}_*$. Moreover, in synthetic data experiments, we set $K = 10, \bar{K} = 15$; in video experiments, we set $K = 5, \bar{K} = 10$ and the underlying rank $r = 2$.

C.2 Training time

Our training time for different matrix sizes, ranks, and outlier densities are reported in Table 4.

Table 4: Training time summary.

Problem settings	Training time
$n = 1000, r = 5, \alpha = 0.1$	1208 secs
$n = 1000, r = 5, \alpha = 0.2$	1209 secs
$n = 1000, r = 5, \alpha = 0.3$	1208 secs
$n = 1000, r = 5, \alpha = 0.1$	1208 secs
$n = 3000, r = 5, \alpha = 0.1$	1615 secs
$n = 5000, r = 5, \alpha = 0.1$	2405 secs
$n = 1000, r = 5, \alpha = 0.1$	1208 secs
$n = 1000, r = 10, \alpha = 0.1$	1236 secs
$n = 1000, r = 15, \alpha = 0.1$	1249 secs

Different from the inference time reported in the main paper, the training was done on a workstation equipped with two Nvidia RTX-3080 GPUs. Note that the training time is not proportional to the problem size due to the high concurrency of GPU computing.

C.3 Visualizations of video background subtraction

In Figure 7, we visualize the results of LRPCA, ScaledGD and AltProj on the task of video background subtraction.

C.4 Generalization

In this section, we study the generalization ability of our model. Specifically, we train our model on small-size and low-rank instances, and test it on instances with larger size or higher rank.

First we train a FRMNN on instances of size 1000×1000 and rank-5. This setting is denoted as the “base” setting. We only train the model once on the base setting and test it on instances with different

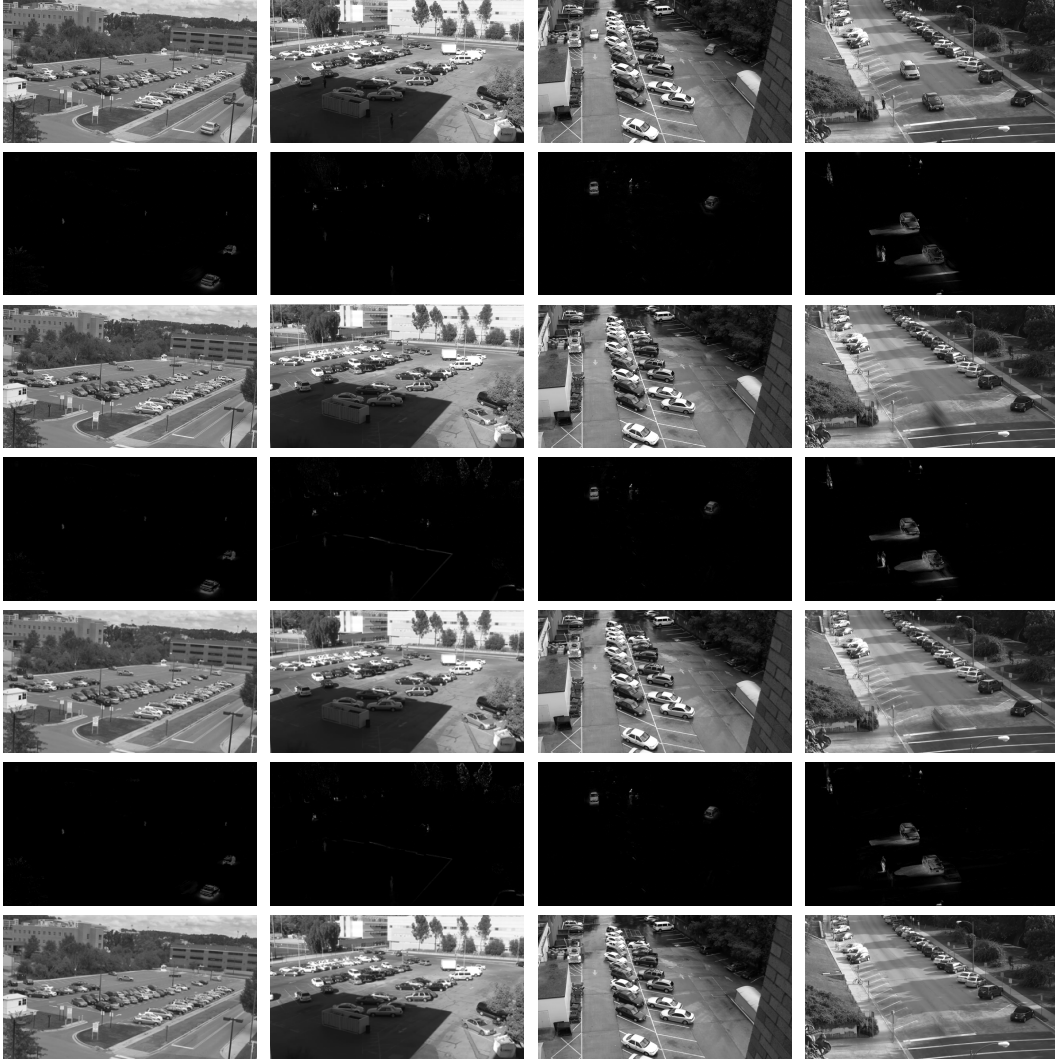


Figure 7: Video background subtraction visual results. Each column represents a selected frame from the tested videos. From left to right, they are ParkingLot1, ParkingLot2, ParkingLot3, and StreetView. The first row contains the original frames. Rows 2 and 3 are the separated foreground and background produced by LRPCA, respectively. Rows 4 and 5 are results for ScaledGD. The last two rows are results for AltProj.

settings (denoted as “target” settings). When we test a model on instances, we use the step sizes $\{\eta_k\}$ directly and scale the thresholdings $\{\zeta_k\}$ by a factor of $(n_{\text{base}}/n_{\text{target}})(r_{\text{target}}/r_{\text{base}})$ due to the ℓ_∞ bound estimation given in Lemma 10.

As a comparison, we also train FRMNNs individually on the target settings. The averaged iterations to achieve 10^{-4} on the testing set are reported in Table 5.

From Table 5, we conclude that our model has good generalization ability w.r.t. n and r . For example, if we train and test a model both with $n = 3000, r = 5$, it takes 6 iterations; if we train a model on the base setting (i.e., $n = 1000, r = 5$) and test it with $n = 3000, r = 5$, it takes 7 iterations. Such generalization of our model works fine with slight loss of performance. That is, a model trained once on the base setting is good enough for larger size or higher rank problems from similar testing sets.

Table 5: Results for generalization test.

Fix $r = 5$, test different n			
Matrix size n	1000	3000	5000
Iterations (Model trained on base setting)	8	7	7
Iterations (Model trained on target setting)	8	6	5
Fix $n = 1000$, test different r			
Matrix rank r	5	10	15
Iterations (Model trained on base setting)	8	10	11
Iterations (Model trained on target setting)	8	8	9

C.5 Analysis of trained parameters

We visualize the trained step sizes and thresholdings in Figures 8 and 9, respectively. Figure 9 demonstrates that the trained thresholdings decay in an exponential rate, which is aligned with our theoretical bound in Lemma 10. Figure 8 shows that η_k takes larger value when k is small. In other words, the algorithm goes very aggressively with large step sizes in the first several steps.

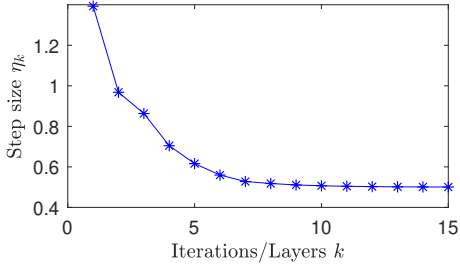


Figure 8: Trained step sizes

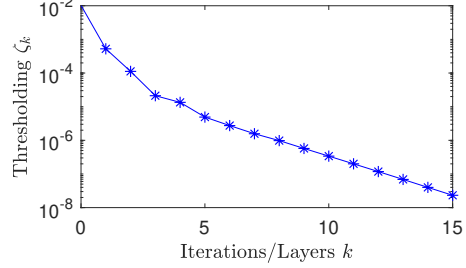


Figure 9: Trained thresholdings